Eserciziario di Programmazione a Oggetti mod. 2

Raccolta organica dei temi d'esame

Prof. Alvise Spanò

a.a. 2017-2025

Ultima revisione: 03/12/2024

Versione 2.1

Indice

1 Tests **4**
1.1 Design Patterns . . . 4
1.2 Riuso di codice e Type system di Java . . . 4
1.3 Specificità e funzionalità Java . . . 7
1.3.1 Equals . . . 8
1.3.2 Compare . . . 9
1.3.3 Programmazione ibrida funzionale . . . 11

2 Esercizi di progettazione e implementazione **12**
2.1 Java . . . 12
2.1.1 Confronto di Collection . . . 12
2.1.2 Thread Pool . . . 12
2.1.3 Piano cartesiano . . . 13
2.1.4 Funzioni di ordine superiore . . . 14
2.1.5 Fattoriale con thread . . . 15
2.1.6 Iteratori asincroni . . . 16
2.1.7 Funzioni . . . 16
2.1.8 Alberi binari di ricerca . . . 17
2.1.9 Alberi binari . . . 18
2.1.10 SkippableArrayList . . . 19
2.1.11 Figure geometriche . . . 19
2.1.12 Artisti . . . 23
2.1.13 Iteratori . . . 25
2.1.14 Sequenza di Fibonacci . . . 25
2.1.15 Classe Random . . . 25
2.1.16 FancyArrayList . . . 26
2.1.17 Minimo e massimo in una lista . . . 26
2.1.18 Hanna Barbera . . . 27
2.2 C++ . . . 28
2.2.1 Pair . . . 28
2.2.2 Matrici . . . 28
2.2.3 Sommatoria . . . 29
2.2.4 Map . . . 29
2.2.5 Funzioni . . . 29
2.2.6 Curve . . . 30
2.2.7 Alberi binari . . . 31
2.2.8 Smart Pointer . . . 31

3 Domande aperte **32**

4 Soluzioni **33**
4.1 Soluzione Tests . . . 33
4.2 Soluzione esercizi di progettazione e implementazione - Java . . . 34
4.2.1 Thread Pool (2.1.2) . . . 34
4.2.2 Piano Cartesiano (2.1.3) . . . 35
4.2.3 Funzioni di ordine superiore (2.1.4) . . . 38
4.2.4 Fattoriale con thread (2.1.5) . . . 40
4.2.5 Iteratori asincroni (2.1.6) . . . 41
4.2.6 Funzioni (2.1.7) . . . 42
4.2.7 Alberi binari di ricerca (2.1.8) . . . 44
4.2.8 Alberi binari (2.1.9) . . . 46
4.2.9 SkippableArrayList (2.1.10) . . . 49
4.2.10 Figure Geometriche (2.1.11) . . . 50
4.2.11 Artisti (2.1.12) . . . 54
4.2.12 Iteratori (2.1.13) . . . 57
4.2.13 Sequenza di Fibonacci (2.1.14) . . . 58

4.2.14 Random (2.1.15) 60
4.2.15 FancyArrayList (2.1.16) 61
4.2.16 Minimo e Massimo di una Lista (2.1.17) 63
4.2.17 Hanna Barbera (2.1.18) 64
4.3 Soluzione esercizi di progettazione e implementazione - C++ 66
4.3.1 Pair (2.2.1) 66
4.3.2 Matrici (2.2.2) 68
4.3.3 Sommatoria (2.2.3) 69
4.3.4 Map (2.2.4) 70
4.3.5 Funzioni (2.2.5) 71
4.3.6 Curve (2.2.6) 73
4.3.7 Alberi binari (2.2.7) 75
4.3.8 Smart Pointer (2.2.8) 82

1 Tests

1.1 Design Patterns

Questa sezione contiene domande sull'argomento design patterns.

1. [*Esame 20/06/2018 Test A*] Si prenda in considerazione il seguente codice Java:

```java
public class MyClass {
    private static MyClass instance;
    public static MyClass get() {
        if (instance == null) instance = new MyClass();
        return instance;
    }
}
```

   Quale design pattern è all'opera?

   ◯ Command ◯ Factory ◯ nessun pattern ☒ Singleton

2. [*Esame 20/06/2018 Test A*] Perché il design pattern denominato *factory* è utile in Java ed in altri linguaggi ad oggetti simili?

   ☒ Perché i costruttori non sono metodi soggetti al *dynamic dispatching*, quindi non è possibile sfruttare il polimorfismo per costruire oggetti se non tramite un metodo non-statico che incapsula la costruzione.

   ◯ Perché la costruzione degli oggetti va nascosta dentro metodi statici al fine di poter rendere privati i costruttori, applicando così i principi di *information hiding* e di incapsulamento tipici della programmazione ad oggetti anche alla creazione di istanze.

   ◯ Siccome le sottoclassi non hanno accesso ai super-costruttori, è necessario dotare le superclassi di metodi wrapper `protected` dei propri costruttori, permettendo alle classi derivate di accedervi indirettamente.

   ◯ Perché non è possibile costruire array con tipi generici in Java, pertanto è preferibile mascherare gli inevitabili cast dentro un metodo statico che funge da wrapper per il costruttore.

1.2 Riuso di codice e Type system di Java

Questa sezione contiene domande sull'argomento polimorfismo, genericità e typing, compresi i temi di *subtyping*, *generics*, *override*, *overloading*, ecc.

1. [*Esame 20/06/2018 Test*] Java versione 10 supporta l'ereditarietà multipla?

   ◯ Sì, ma solo tra classi, mentre è possibile implementare solamente una interfaccia alla volta.

   ◯ No, nemmeno tra interfacce, a causa del noto *problema del diamante*.

   ◯ Sì, perché risolve il *problema del diamante* mescolando opportunamente le *virtual table* delle superclassi secondo l'ordine di apparizione di queste ultime dopo la keyword `extends`.

   ☒ No, però non c'è limite al numero di interfacce che una classe può implementare.

2. [*Esame 24/05/2018 Test*] In generale, a cosa servono le due forme di polimorfismo (subtyping e generics) che offre Java oggi?

   ☒ A riusare lo stesso codice, ovvero chiamare un metodo, con parametri di tipo differente.

   ◯ A definire classi che implementano due o più interfacce.

   ◯ A riusare lo stesso codice, in termini di classi e dei loro membri, ereditandoli anziché riscrivendoli.

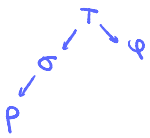
   ◯ A permettere di definire classi o interfacce parametriche su altri tipi.

3. [*Esame 24/05/2018 Test*] Si assuma la relazione di subtyping $\preceq$ (ed il suo complemento $\not\preceq$) tale per cui, dati due tipi A e B, se $A \preceq B$ allora A è *sottotipo* di B. E siano $\tau, \sigma, \rho, \varphi$ tipi concreti per cui valgono le seguenti relazioni: $\sigma \preceq \tau$, $\rho \preceq \sigma$ e $\varphi \preceq \tau$. Si indichi quali delle seguenti relazioni *non* è valida.

   ◯ $\rho \preceq \tau$

   ◯ $\sigma \not\preceq \rho$

   ☒ $\varphi \preceq \rho$



- ◯ $\tau \not\preceq \varphi$

4. [*Esame 24/05/2018 Test*] L'ereditarietà è una forma di polimorfismo?
   - ◯ Sì, perché permette il riuso del codice di una classe e dei suoi membri pubblici anche nelle sottoclassi.
   - ☒ No, ma rende possibile il polimorfismo subtype poiché garantisce che i membri pubblici delle superclassi esistano nelle sottoclassi.
   - ◯ Si, perché, grazie al dynamic dispatching, viene garantita l'invocazione corretta a run-time dei metodi in override nelle sottoclassi anche quando i tipi a compile-time sono soggetti a *subsumption*.
   - ◯ No, perché l'ereditarietà non garantisce davvero che i membri pubblici delle superclassi esistano anche nelle sottoclassi, in quanto è possibile cambiare la visibilità dei membri pubblici ereditati al momento dell'override, ad esempio rendendoli privati, quindi non invocabili.

5. [*Esame 24/05/2018 Test*] Quali parti della firma di un metodo sono coinvolte nella risoluzione dell'overloading in Java?
   - ◯ Il tipo ed il numero dei parametri, il tipo di ritorno ma non le eccezioni dichiarate.
   - ◯ Il tipo ed il numero dei parametri, il tipo di ritorno ed anche le eccezioni dichiarate.
   - ☒ Il tipo ed il numero dei parametri e basta, senza tipo di ritorno né eccezioni.
   - ◯ Il tipo ed il numero dei parametri inizialmente; se sono ambigui anche il tipo di ritorno, ma mai le eccezioni.

6. [*Esame 30/05/2019 Test*] Si prenda in considerazione il seguente codice Java 8+ contenente una semplice gerarchia di classi ed un metodo generico di nome `map`:

```java
public class Rpg {
    public static <A, B> Collection<B> map(Collection<A> c, Function<A, B> f) {
        List<B> r = new ArrayList<>();
        for (A a : c) {
            r.add(f.apply(a));
        }
        return r;
    }

    public static abstract class Character<R extends Number> {
        public int level;
        public final String name;
        protected Character(int level, String name) {
            this.level = level;
            this.name = name;
        }
        public abstract R attack();
    }

    public static class Paladin extends Character<Float> {
        public float mana;
        public Paladin(int level, String name) {
            super(level, name);
            mana = 100.f;
        }
        @Override
        public Float attack() { return mana * level / 2.f; }
    }

    public static class Rogue extends Character<Number> {
        public int energy;
        public Rogue(int level, String name) {
            super(level, name);
            energy = 50;
```

```
        }
        @Override
        public Integer attack() { return (energy -= 35) > 20 ? level * 2 : 0; }
    }
}

```

Gli *override* del metodo `attack()` sono validi oppure no?

- ◯ No, perché le sottoclassi `Paladin` e `Rogue`, specializzando il tipo di ritorno rispetto al type argument passato alla superclasse, violano la regola della contro-varianza del tipo di ritorno che Java supporta.
- ☒ Sì, anche se le sottoclassi `Paladin` e `Rogue` specializzano il tipo di ritorno rispetto al type argument passato alla superclasse, la specializzazione del tipo di ritorno è supportata dall'overriding in Java.
- ◯ Non quello della classe `Rogue`, poiché passa `Number` come type argument alla superclasse, ma dichiara `Integer` (che è un sottotipo) come tipo di ritorno dell'override.
- ◯ Sì, perché la *type erasure* converte il generic `R` nel suo constraint `Number` ed i tipi di ritorno si possono specializzare rispetto ad esso, seguendo la regola della controvarianza.

7. [*Esame 30/05/2019 Test*] In relazione al codice della domanda 6 (Sezione 1.2), si consideri ora il seguente codice *main*:

```
List<Paladin> retadins = new ArrayList<>();
retadins.add(new Paladin(60, "Leeroy Jenkins"));
retadins.add(new Paladin(80, "Arthas"));
```

Affinché il seguente statement compili, quali modifiche alla firma della funzione `map()` sarebbero necessarie e sufficienti?

```
Collection<Number> r1 = map(retadins, new Function<Character, Float>() {
    @Override
    public Float apply(Character c) {
        return 1.3f * (float) c.attack();
    }
});
```

- ☒ `static <A, B> Collection<B> map(Collection<A> c, Function<? super A, ? extends B> f)`
- ◯ `static <A, B> Collection<B> map(Collection<A> c, Function<? extends A, ? super B> f)`
- ◯ `static <A, B> Collection<B> map(Collection<? super A> c, Function<A, B> f)`
- ◯ `static <A, B> Collection<B> map(Collection<? extends A> c, Function<A, ? extends B> f)`

8. [*Esame 30/05/2019 Test*] In relazione al codice della domanda 6 (Sezione 1.2), si prendano ora in considerazione i due seguenti metodi in overload:

```
public static int normalizeAttack(Character c) { return 1 + (int) c.attack(); }

public static Float normalizeAttack(Paladin c) { return c.attack(); }
```

(a) Sarebbero validi se aggiunti come membri alla classe `Rpg`?

- ◯ Sì, perché il tipo di ritorno discrimina univocamente la risoluzione dell'overloading per il compilatore.
- ◯ No, perché i metodi statici non supportano l'overloading.
- ◯ Sì, perché i due metodi hanno parametri di tipo differente; e nonostante `Character` sia un supertipo di `Paladin` il compilatore è in grado di discriminare.
- ◯ No, perché sebbene i due metodi abbiano parametri di tipo differente, `Character` è un supertipo di `Paladin`, quindi il compilatore non sarebbe in grado di risolverre l'overloading senza ambiguità.

(b) Il seguente statement compilerebbe, assumendo di conservare la firma originale della funzione `map()`? E se sì, quale overload di `normalizeAttack()` verrebbe passato come *method reference*?

```
Collection<Number> r2 = map(retadins, Rpg::normalizeAttack);
```

- ◯ Sì, compilerebbe e risolverebbe `normalizeAttack(Paladin)`.
- ◯ Sì, compilerebbe ed risolverebbe `normalizeAttack(Character)`.

- ◯ No, non compilerebbe senza modificare la firma della `map()`.
- ◯ No, non compilerebbe comunque perché nessuno dei due overload ha tipo di ritorno `Number`.

9. [*Esame 31/01/2020*] Si prendano in considerazione le seguenti interfacce Java:

```java
public interface Solid extends Comparable<Solid> {
    double area();
    double volume();
    PositionedSolid at(Point origin);

    default int compareTo(Solid s) {
        Function<S, Double> f = (x) -> x.volume();
        return Double.compare(f.apply(this), f.apply(s))
    }
}

public interface Polyhedron extends Solid {
    double perimeter();
    @Override
    PositionedPolyhedron at(Point origin);
}

public interface PositionedSolid {
    Point origin();
}

public interface PositionedPolyhedron extends PositionedSolid, Iterable<Point> {}
```

Quale caratteristica del linguaggio Java è all'opera nell'override del metodo `at()` all'interno dell'interfaccia `Polyhedron`?

- ◯ E' un overload, non un override.
- ◯ L'overriding supporta la *co-varianza* del tipo di ritorno di un metodo.
- ◯ L'overriding supporta la *contro-varianza* del tipo di ritorno di un metodo.
- ◯ Un override può cambiare liberamente il tipo di ritorno di un metodo.

10. [*Esame 04/09/2018*] Si prenda in consederazione il seguentre codice:

```java
public class Random {
    public Random() { ... }
    public Random(int seed) { ... }
    public boolean nextBoolean() { ... }
    public int nextInt() { ... }
    public double nextDouble() { ... }
}
```

I metodi che essa presenta hanno nomi diversi: sarebbe stato possibile definirli tutti con lo stesso nome (ad esempio `next()`) mantenendo inalterate le firme e la semantica?

- ◯ Sì: l'overloading sarebbe possibile e permetterebbe anche l'invocazione polimorfa di `next()` con risoluzione dipendente dal contesto;
- ◯ Non è necessario: Java offre una forma speciale di risoluzione dipendente dal contesto per metodi aventi nome simile e firma differente solo per il tipo di ritorno;
- ◯ No: l'overloading non è possibile tra metodi che differiscono solamente per il tipo di ritorno;
- ◯ Sì: l'overloading sarebbe possibile, tuttavia non darebbe alcun beneficio pratico poiché Java ad oggi non consente la risoluzione dell'overloading dipendente dal contesto.

1.3 Specificità e funzionalità Java

Questa sezione contiene domande su interfacce, metodi, costrutti esistenti nelle Java API e funzionalità specifiche dalle release Java, come ad esempio programmazione ibrida funzionale, ecc.

1.3.1 Equals

1. Si prenda in considerazione il seguente codice Java:

```java
public interface Equatable<T> {
    boolean equalsTo(T x);
}

public static class Person<P extends Person<P>> implements Equatable<P> {

    public final String name;
    public final int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o)
            return true;
        if (o == null)
            return false;
        if (getClass() != o.getClass())
            return false;

        return equalsTo((P) o);
    }

    @Override
    public boolean equalsTo(P other) {
        return name.equals(other.name) && age == other.age;
    }

    @Override
    public String toString() {
        return name;
    }

}

public class Artist extends Person<Artist> {
    public final Hair hair;

    public Artist(String name, int age, Hair hair) {
        super(name, age);
        this.hair = hair;
    }

    @Override
    public boolean equalsTo(Artist other) {
        return super.equalsTo(other) && hair.equals(other.hair);
    }
}

public class Hair implements Equatable<Hair> {
    public final int length;
```

```java
    public final Set<Color> colors;

    public Hair(int length, Set<Color> colors) {
        this.colors = colors;
        this.length = length;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o)
            return true;
        if (o == null)
            return false;
        if (getClass() != o.getClass())
            return false;
        return equalsTo((Hair) o);
    }

    // 1.c
    @Override
    public boolean equalsTo(Hair x) {
        return colors.equals(x.colors) && length == x.length;
    }
}

public enum Color {
    BROWN, DARK, BLONDE, RED, GRAY;
}

```

Si considerino ora i seguenti binding in Java:

```java
Person alice = new Person("Alice", 23),
david = new Artist("Bowie", 69, new Hair(75, Set.of(Color.RED, Color.BROWN, Color.GRAY)));
Artist morgan = new Artist("Morgan", 47, new Hair(20, Set.of(Color.GRAY, Color.DARK))),
madonna = new Artist("Madonna", 60, new Hair(50, Set.of(Color.BLONDE)));
```

Per ciascuna delle seguenti espressioni Java si indichi con una crocetta l'esito della computazione - se produce un booleano, se non compila oppure se lancia un'eccezione a runtime:

a.	`alice.equals(null)`	◯ Si	◯ No
b.	`alice.equals(alice)`	◯ Si	◯ No
c.	`null.equals(david)`	◯ Si	◯ No
d.	`alice.equals(david)`	◯ Si	◯ No
e.	`alice.equalsTo(morgan)`	◯ Si	◯ No
f.	`morgan.equals(morgan)`	◯ Si	◯ No
g.	`morgan.equals(madonna)`	◯ Si	◯ No
h.	`morgan.equals(david)`	◯ Si	◯ No
i.	`morgan.equalsTo(david)`	◯ Si	◯ No
j.	`david.equalsTo(morgan)`	◯ Si	◯ No
k.	`madonna.equals(3)`	◯ Si	◯ No
l.	`madonna.equalsTo("Madonna")`	◯ Si	◯ No

1.3.2 Compare

1. [*Esame 20/06/2018 Test A*] Si assuma la firma del seguente metodo che ordina (stabile, crescente) una lista di elementi confrontabili:

```java
static <T extends Comparable<T>> void sort(List<T> l)
```

(a) Se ipoteticamente definissimo un altro metodo `sort`, in overload con il precedente, avente firma:

```
static <T> void sort(List<? extends Comparable<T>> l)
```

esso rappresenterebbe un caso di overloading valido?

- ◯ Sì, perché i constraint del generic `T` sono diversi, quindi il compilatore sarebbe in grado di disambiguare.
- ◯ No, perché i metodi avrebbero la medesima *type erasure*.
- ◯ Sì, perché anche se l'overload è ambiguo per il compilatore, a runtime viene invocato il metodo giusto grazie al *dynamic dispatching*.
- ◯ No, perché qualunque overload che coinvolge generics sarebbe ambiguo a causa della *type erasure*.

Si considerino ora le seguenti classi:

```
1  public class Humanoid implements Comparable<Humanoid> {
2      protected int strength;
3
4      public Humanoid(int strength) { this.strength = strength; }
5
6      @Override
7      public int compareTo(Humanoid o) { return -(strength - o.strength); }
8  }
9
10 public class Elf extends Humanoid {
11     protected int mana;
12
13     public Elf(int strength, int mana) { super(strength); this.mana = mana; }
14
15     @Override
16     public int compareTo(Humanoid o) {
17         if (o instanceof Elf) {
18             Elf e = (Elf) o;
19             return -((mana + strength) - (e.mana + e.strength));
20         }
21         else return super.compareTo(o);
22     }
23 }
```

(b) Ipotizzando il codice seguente, la chiamata `sort(l)` sarebbe accettata dal compilatore Java?

```
1  List<Elf> l = new ArrayList<Elf>();
2  l.add(new Elf(11, 23));
3  sort(l);
```

- ◯ No: il constraint `<T extends Comparable<T>>` impone che `Elf` implementi `Comparable<Elf>`, ma invece implementa `Comparable<Humanoid>`.
- ◯ Sì: anche se il constraint `<T extends Comparable<T>>` impone che `Elf` implementi `Comparable<Elf>`, la chiamata risulta compatibile con `Comparable<Humanoid>` perché `Elf` è un sottotipo di `Humanoid`, da cui segue che `Comparable<Elf>` è sottotipo di `Comparable<Humanoid>`.
- ◯ No: sebbene il tipo dell'argomento sussuma in `List<Elf>`, l'interfaccia `List` non estende l'interfaccia `Comparable`.
- ◯ Sì: la chiamata sussume il tipo dell'argomento in `List<Humanoid>`, che soddisfa il constraint `<T extends Comparable<T>>` perché `Humanoid` implementa `Comparable<Humanoid>`.

(c) Si consideri un altro estratto di codice:

```
1  List<Humanoid> l = new ArrayList<>();
2  Humanoid a = new Elf(10, 8), b = new Humanoid(8), c = new Humanoid(12);
3  l.add(a); l.add(b); l.add(c);
4  sort(l);
```

Quale tra i seguenti identificatori è legato all'ultimo elemento della lista ordinata dopo la chiamata `sort(l)`?

◯ `a` ◯ `b` ◯ `c` ◯ non compila

2. Si prenda in considerazione il codice della domanda 1 (Sezione 1.3.1), il seguente statement sarebbe accettato dal compilatore Java 7+?

```
1
2         Artist c = Collections.max(artists, new Comparator<Person>() {
3                 public int compare(Person a, Person b) {
4                         return a.age - b.age;
5                 }
6         });
```

- ◯ No: il primo argomento `artists` ha tipo `List<Artist>`, istanziando il generic `T` con `Artist`, pertanto l'oggetto di tipo `Comparator<Person>` passato come secondo argomento non soddisfa il wildcard perché `Person` è diverso da `Artist`.
- ◯ Sì: il primo argomento `artists` ha tipo `List<Artist>`, istanziando il generic `T` con `Artist`, pertanto l'oggetto di tipo `Comparator<Person>` passato come secondo argomento soddisfa il wildcard con *lower bound* perché `Person` è supertipo di `Artist`.
- ◯ No: sebbene il primo argomento `artists` abbia tipo `List<Artist>`, il generic `T` viene istanziato col tipo `Person` in modo da soddisfare il wildcard con *upper bound*, tuttavia il tipo di ritorno `Person` non può essere *sussunto* in `Artist`.
- ◯ Sì: il primo argomento `artists` ha tipo `List<Artist>`, ma grazie al wildcard con *upper bound* il generic `T` viene istanziato col tipo `Person`; il compilatore tuttavia tiene traccia dei tipi degli oggetti a runtime, quindi il tipo di ritorno in realtà è `Artist`.

1.3.3 Programmazione ibrida funzionale

1. Si prendano in considerazione le seguenti interfacce Java:

```
1  public interface Solid extends Comparable<Solid> {
2      double area();
3      double volume();
4      PositionedSolid at(Point origin);
5
6      static <S extends Solid> int compareBy(Function<S, Double> f, S s1, S s2) {
7          return Double.compare(f.apply(s1), f.apply(s2));
8      }
9      static <S extends Solid> Comparator<S> comparatorBy(Function<S, Double> f) {
10         return (s1, s2) -> compareBy(f, s1, s2);
11     }
12     default int compareTo(Solid s) {
13         return compareBy((x) -> x.volume(), this, s);
14     }
15 }
```

La lambda espressione `(x) -> x.volume()` passata come primo argomento al metodo `compareBy()` nel corpo del metodo `compareTo()`, avente tipo `Function<Solid, Double>`, è equivalente a quale dei seguenti costrutti del linguaggio Java?

- ◯ Ad un riferimento ad un metodo statico (*static method reference*) della classe `Solid`; ovvero all'espressione `Solid::volume`.
- ◯ Ad un riferimento ad un metodo non-statico (*instance method reference*) di un oggetto arbitrario di tipo `Solid`; ovvero all'espressione `Solid::volume`.
- ◯ Ad un riferimento ad un metodo non-statico (*instance method reference*) di un oggetto specifico, che è `this` in questo caso; ovvero all'espressione `this::volume`.
- ◯ A nessuna dei precedenti.

2 Esercizi di progettazione e implementazione

I seguenti esercizi servono a verificare la capacità dello studente di progettare e implementare soluzioni utilizzando il paradigma a oggetti, con i linguaggi Java e C++, nelle rispettive sezioni.

2.1 Java

2.1.1 Confronto di Collection

[*Esame* ***10/09/2024*** *– 04/09/2018*]

1. Si scriva in linguaggio Java 8+ la firma e l'implementazione di un metodo statico e generico `compareMany()` che confronta due collection. Dati due argomenti `c1` e `c2` aventi tipo `Collection<A>` e `Collection<B>` (dove `A` e `B` sono i due *type parameter* del metodo), esso confronta ogni elemento di tipo `A` della prima collection con l'elemento di tipo `B` alla stessa posizione della seconda. Il confronto degli elementi deve avvenire chiamando il metodo `compareTo()` opportunamente: questo implica che ci sono dei vincoli sui generics `A` e/o `B`. Il risultato del metodo `compareMany()` è di tipo `int` e deve rispettare la semantica del confronto cosiddetto *a tre vie*. Si implementi la semantica del confronto nel seguente modo:

   - `c1` e `c2` sono uguali se e solo se le lunghezze sono uguali e tutti gli elementi sono uguali;
   - `c1` è minore di `c2` se la lunghezza di `c1` è minore o uguale a quella di `c2` e se tutti gli elementi di `c1` sono minori o uguali ai corrispettivi elementi di `c2`;
   - altrimenti `c1` è maggiore di `c2`.

2.1.2 Thread Pool

[*Esame 10/09/2024 –* ***13/01/2023*** *– 22/01/2019*] Si prenda in considerazione la seguente interfaccia Java:

```java
public interface Pool<T, R> {
    void add(T x);                          // popola la pool con un nuovo elemento
    R acquire() throws InterruptedException;  // acquisisce una risorsa
    void release(R x);                      // rilascia una risorsa e la rimette nella pool
}
```

Una pool è un container di oggetti che si comporta come una coda bloccante: è possibile ottenere una risorsa con `acquire()` per poi restituirla alla pool tramite il metodo `release()`.

Il generic `T` astrae il tipo degli oggetti contenuti internamente nella pool, mentre `R` è il tipo della risorsa restituita dalla `acquire()`: il motivo per cui sono due generic differenti è per consentire alle classi che implementano questa interfaccia di rappresentare in modo diverso, se necessario, gli elementi conservati all'interno e le risorse restituite dalla `acquire()`.

Quando la coda è vuota e nessun oggetto è disponibile, il metodo `acquire()` deve essere bloccante: al fine di semplificare l'implementazione, si utilizzi un oggetto di tipo `LinkedBlockingQueue` come campo interno. Riportiamo un estratto della classe `LinkedBlockingQueue` definita dal JDK con i metodi pubblici più significativi:

```java
class LinkedBlockingQueue<E> implements BlockingQueue<E> {
    LinkedBlockingQueue();                  // costruttore
    void add(E x);                          // aggiunge un elemento
    E take() throws InterruptedException;   // estrae la testa (bloccante)
    E peek();                               // ritorna la testa senza rimuoverla,
                                            // oppure null se vuota

    int size();                             // numero di elementi
    // etc...
}
```

1. Si definisca una interfaccia `BasicPool` che estende l'interfaccia `Pool` e che ha un solo generic per rappresentare sia il tipo degli elementi interni sia il tipo delle risorse restituite dalla `acquire()`.

2. Si implementi una classe `SimplePool` che implementa l'interfaccia `BasicPool` e realizza una semplice coda bloccante.

3. Si implementi una classe `AutoPool` che implementa l'interfaccia `Pool` realizzando un meccanismo di *auto-release* delle risorse. Se `p` è una pool, l'obiettivo è fare in modo che una risorsa `x` acquisita tramite la chiamata `p.acquire()` non debba essere esplicitamente riconsegnata alla pool chiamando `p.release(x)`, ma sia possibile rilasciarla invocando `x.release()`.

   Suggerimento: si definisca un nuovo tipo parametrico `Resource` che si comporta come un *proxy* per l'oggetto contenuto al suo interno e che implementa la logica di *auto-release* rilasciando `this`.

4. Si automatizzi il meccanismo di cui sopra facendo in modo che un oggetto di tipo `Resource` si rilasci *autonomamente* quando non esistono più riferimenti ad esso.

   Suggerimento: la superclasse `Object` definisce un metodo `finalize()` che viene invocato nel momento in cui l'oggetto viene cancellato dal garbage collector.

2.1.3 Piano cartesiano

[*Esame 25/06/2024 – **06/09/2019***]

1. Si implementi in Java 8+ un sistema di classi che rappresentano punti, linee e poligoni regolari nel piano cartesiano $\mathbb{R} \times \mathbb{R}$.

   (a) Si implementi una classe di nome `Point` che rappresenta punti bidimensionali immutabili, in cui le coordinate `x` ed `y` sono di tipo `double`.

   (b) Si implementi una classe di nome `Segment` che rappresenta segmenti bidimensionali immutabili, il cui costruttore prende due argomenti di tipo `Point`. Essa deve fornire un metodo `length()` che restituisce la lunghezza del segmento calcolando la distanza euclidea tra i due punti.

   Si implementi un tipo che rappresenta segmenti bidimensionali immutabili, ovvero una classe `Line` il cui costruttore prende due argomenti di tipo `Point`. Essa deve fornire un metodo `length()` che ne calcola la lunghezza tramite la distanza euclidea tra i due punti.

   (c) Si prenda in considerazione la seguente classe astratta `Polygon` che rappresenta poligoni regolari come liste di punti (minimo 3, verificato a runtime). I punti nella lista determinano l'ordine di costruzione dei segmenti di cui è composto il poligono. Ad esempio, una lista contenente i seguenti 3 punti $A = (0, 0)$, $B = (3, 3)$ e $C = (3, 0)$ rappresenta un triangolo rettangolo in cui il primo lato è AB, il secondo è BC ed il terzo è CA.

```java
public abstract class Polygon {
    protected final List<Point> points;
    protected Polygon(List<Point> points) {
        assert points.size() >= 3;
        this.points = points;
    }
    public Iterator<Line> lineIterator() { /* da implementare */ }
    public double perimeter() { /* da implementare */ }
    public abstract double area();
}
```

   i. Per quale motivo è necessario vincolare a runtime la dimensione minima della lista di punti tramite un `assert` anziché sfruttare in qualche modo il type system per fare un controllo statico? Si articoli una breve risposta.

   ii. Si implementi il metodo `lineIterator()` che costruisce un iteratore su oggetti di tipo `Line` e si comporta come un *wrapper* dell'iteratore estratto dal campo `points`. Gli oggetti prodotti dall'iteratore di `Line` devono essere costruiti *dinamicamente* leggendo coppie di punti adiacenti dall'iteratore di `Point`. Si implementi una logica di *caching* dell'ultimo punto letto per permettere la costruzione di un nuovo segmento adiacente all'ultimo ad ogni invocazione del metodo `next()`. Si badi inoltre a riusare opportunamente il primo punto come secondo estremo dell'ultimo segmento costruito.

   iii. Si implementi il metodo `perimeter()` in funzione del metodo `lineIterator()`, ovvero calcolando il perimetro del poligono iterando sui segmenti che lo compongono.

2. Estendiamo ora la gerarchia di classi introducendo tipi specializzati per i poligoni classici.

   (a) Si implementi una sottoclasse di `Polygon` di nome `Triangle` che rappresenta triangoli qualunque.

```java
public class Triangle extends Polygon {
    public Triangle(Point p1, Point p2, Point p3) { /* da implementare */ }
    @Override
    public double area() { /* da implementare */ }
}
```

Il costruttore prende 3 argomenti di tipo `Point` e deve chiamare il super-costruttore opportunamente. Si implementi il metodo `area()` in modo che calcoli l'area del triangolo senza fare assunzioni sulla sua forma.

(b) Si implementi un tipo che rappresenta triangoli rettangoli tramite una sottoclasse di `Triangle`. Si badi in particolar modo a definire un costruttore che sintetizzi al massimo le informazioni passate come argomenti. Si prediligano i controlli statici a quelli dinamici, se possibile, e si tenti di minimizzare i casi di ambiguità o gli stati di invalidità dell'oggetto; nel caso in cui si ritenga necessario fare dei controlli a runtime, si usi il costrutto `assert`.

(c) Si implementi una sottoclasse di `Polygon` di nome `Rectangle` che rappresenta rettangoli.

```java
public class Rectangle extends Polygon {
    public Rectangle(Point p1, Point p3) { /* da implementare */ }
    @Override
    public double area() { /* da implementare */ }
}
```

Il costruttore prende i 2 punti della diagonale e deve passare al super-costruttore i 4 punti che costituiscono il rettangolo calcolandone le coordinate per proiezione ortogonale. Si badi all'ordine in cui i punti compaiono nella lista, affinché siano adiacenti e permettano di comporre i lati correttamente. Si implementi il metodo `area()` in modo che calcoli l'area del rettangolo.

(d) Si implementi una sottoclasse di `Rectangle` di nome `Square` che rappresenta quadrati.

```java
public static class Square extends Rectangle {
    public Square(Point p1, double side) { /* da implementare */ }
}
```

Il costruttore prende il punto in basso a sinistra e la dimensione del lato e deve chiamare il super-costruttore opportunamente.

3. Si prenda in considerazione il seguente codice, in cui compare l'invocazione di un metodo statico `max` da definire:

```java
Square sq1 = new Square(new Point(10., -4.), 0.1),
       sq2 = new Square(new Point(1., 20.), 0.01);
Collection<Square> squares = List.of(sq1, sq2);
Rectangle r = max(squares, new Comparator<Polygon>() {
    @Override
    public int compare(Polygon a, Polygon b) {
        return (int) (a.area() - b.area());
    }
});
```

(a) Si scriva la firma e l'implementazione del metodo statico `max()` invocato nell'ultimo statement affinché il codice di cui sopra compili correttamente. Si renda tale metodo più generico possibile e non monomorfo rispetto ai tipi che compaiono in questa invocazione, prestando particolare attenzione ai vincoli sui generics. La semantica di `max` è facilmente intuibile: trova l'elemento maggiore usando il comparatore per confrontare gli elementi della collection di input.

(b) Assumendo il comportamento corretto del metodo `max()`, quale delle seguenti espressioni booleane è vera?
◯ `sq1 == r` ◯ `sq2 == r` ◯ `r == null` ◯ nessuna delle precedenti

(c) Quale dei seguenti numeri razionali rappresenta il valore di tipo `double` computato dall'espressione `r.area()`?
◯ 10^{-1} ◯ 10^{-2} ◯ 10^{-4} ◯ non è un quadrato ma un rettangolo

2.1.4 Funzioni di ordine superiore

[*Esame 14/06/2024*]

1. Si implementino le seguenti funzioni di ordine superiore in Java 8+.

(a) La prima è una variante della classica funzione `map()`[1] che opera su iteratori anziché su collection. L'iteratore restituito in output deve applicare la funzione `f` a ciascun elemento di tipo `A` fornito dall'iteratore `it`, producendo oggetti di tipo `B`.

```
static <A, B> Iterator<B> mapIterator(Iterator<A> it, Function<A, B> f)
```

(b) La seconda funzione di ordine superiore è semanticamente equivalente al costrutto for-each: ad ogni oggetto di tipo `T` nell'iterable in input viene applicato il consumer `f`.

```
static <T> void forEach(Iterable<T> it, Consumer<T> f)
```

(c) Si implementi un tipo `Pair` parametrico su due tipi `A` e `B`. Si scelga liberamente se fornire una implementazione mutabile o immutabile.

(d) Si prenda ora in considerazione la seguente firma di funzione:

```
static <A, B> Iterator<B> applyFuns(Iterable<Pair<Function<A, B>, A>> l)
```

Per ogni coppia dell'iterable in input, essa deve applicare la funzione che si trova nella prima componente della coppia all'oggetto di tipo `A` che si trova nella seconda componente. Si implementi la suddetta funzione tramite **una singola invocazione** della `mapIterator()` di cui al punto (a).

(e) Si prenda ora in considerazione la seguente firma di funzione:

```
static <T> void acceptFuns(Iterable<Pair<Consumer<T>, T>> l)
```

Per ogni coppia dell'iterable in input, essa deve applicare la funzione consumer che si trova nella prima componente della coppia all'oggetto di tipo `T` che si trova nella seconda componente. Si implementi la suddetta funzione **tramite una singola invocazione** della `forEach()` di cui al punto (b).

(f) Con riferimento al punto (a) si consideri la seguente firma di funzione:

```
static <A, B> Iterator<Supplier<B>> asyncMapIterator(Iterator<A> it, Function<A, B> f)
```

Essa consiste in una variante asincrona della `mapIterator()` in cui l'applicazione della funzione `f` ad ogni elemento di tipo `A` prodotto dall'iteratore in input deve avere luogo ogni volta in un thread nuovo. In altre parole, ogni computazione della funzione `f` deve avvenire concorrentemente. Il risultato di ciascuna computazione, naturalmente, non può essere ritornato subito dalla `next()` dall'iteratore in output, altrimenti sarebbe necessario attendere la fine di ciascun thread, vanificando ogni forma di concorrenza. Per questo motivo l'iteratore in output produce `Supplier<B>` anziché oggetti di tipo `B`. Sono i `Supplier`[2] che devono attendere la fine dell'esecuzione dei thread.

Suggerimento: l'implementazione è piuttosto contenuta, non servono datatype di appoggio né metodi ausil iari. Si sfrutti al massimo lo scoping, in particolare le chiusure delle lambda (o delle anonymous class, se si preferisce).

(g) Con riferimento al punto (b) si consideri la seguente firma di funzione:

```
static <T> void asyncForEach(Iterable<T> it, Consumer<T> f)
```

Questa è una variante asincrona della `forEach()`: l'applicazione della funzione `f` ad ogni elemento di tipo `T` contenuto nell'iterable in input deve avere luogo ogni volta in un thread nuovo. In altre parole, ogni computazione della funzione `f` deve avvenire concorrentemente.

Suggerimento: poiché i `Consumer` non hanno risultato, non si pone il problema di attendere la fine delle computazioni dei thread.

2.1.5 Fattoriale con thread

[*Esame* ***10/01/2024*** *– 30/05/2019*]

1. Si consideri un metodo statico in Java 8+ di nome `parallelFactorial()` che data una `Collection<Integer>` produce una `Collection<FactorialThread>`. Per ogni intero della collection di input viene effettuato lo *spawning* di un nuovo thread che ne computa il fattoriale e ne conserva il risultato in qualche modo. Tutti i thread creati vengono ritornati nella collection di output, che naturalmente ha la stessa lunghezza della collection di input.

   (a) Si implementi la classe `FactorialThread` in modo che estenda `java.lang.Thread` e fornisca un metodo getter per l'accesso al risultato della computazione del fattoriale. Si presti attenzione all'attesa della fine della computazione, gestendo opportunamente la cosa.

[1] Conosciuta anche con il nome di `transform()` in certe librerie.

[2] Si rammenti che un `Supplier<T>` non è altro che una *functional interface* con un solo metodo `get()` che non ha parametri e ritorna un oggetto di tipo `T`.

(b) Si implementi il metodo statico `parallelFactorial()` avente la seguente firma:

```
static Collection<FactorialThread> parallelFactorial(Collection<Integer> c)
```

Ogni thread deve lavorare concorrentemente agli altri, ciascuno calcolando il fattoriale di un intero proveniente dalla collection di input. La funzione `parallelFactorial()` non deve attendere la fine delle computazioni, ma deve ritornare subito la collection di output.

(c) Si raffini la firma del metodo statico `parallelFactorial()` di cui al punto precedente in modo che il tipo di input ed il tipo di output siano, rispettivamente, più generale e più specializzato possibile, senza tuttavia rivelare dettagli sull'implementazione.

(d) Si scriva uno snippet di codice che testi il metodo statico `parallelFactorial()` stampando i risultati di ciascuna computazione.

(e) Si dia una seconda implementazione del metodo statico `parallelFactorial()` usando la funzione di ordine superiore map()[3]. In particolare si proceda nel seguente modo:

i. Si implementi un metodo statico `map()` avente la seguente firma:

```
static <A, B> List<B> map(Iterable<A> i, Function<A, B> f)
```

Esso deve applicare la funzione `f` ad ogni elemento di `i` e produrre in output tutti i risultati delle applicazioni.

ii. Si reimplementi `parallelFactorial()` tramite una singola invocazione della `map()`.

2.1.6 Iteratori asincroni

[*Esame 05/09/2023*]

1. Si implementi un metodo statico generico in Java 8+ che, dato un iteratore ed una funzione, produce un nuovo iteratore che in maniera asincrona applica la funzione ad ogni elemento dell'iteratore originale. Ciò significa che un thread diverso deve processare ciascun elemento.

(a) Si implementi tutto ciò che è necessario dello snippet seguente.

```
static <A, B> Iterator<Supplier<B>> asyncIterator(Iterator<A> it, Function<A, B> f) {
    return new Iterator<>() {
        @Override
        public boolean hasNext() { /* da implemetare */ }

        private class Future implements Supplier<B> {
            public Future(Supplier<B> f) { /* da implementare */ }

            /* da completare/implementare */
        }

        @Override
        public Supplier<B> next() { /* da implementare */ }
    };
}
```

Si badi che la classe innestata `Future` ha lo scopo di semplificare l'implementazione della `next()`. Essa rappresenta una computazione in corso che non è ancora terminata. Essendo di fatto un `Supplier`, sarà possibile conoscere il risultato invocando il metodo `get()`. Naturalmente questo risultato sarà possibile ottenerlo solamente dopo aver atteso che il relativo thread abbia finito di computare il `Supplier` passato in costruzione.

(b) Si implementi il seguente metodo statico in modo che utilizzi il metodo statico `asyncIterator()` di cui all'esercizio precedente per scorrere l'argomento iterabile e ordinare in maniera asincrona i suoi elementi[4].

2.1.7 Funzioni

[*Esame 30/06/2023*]

[3] La funzione `map()` qui richiesta è equivalente a quella che JDK e STL chiamano `transform()`.
[4] Si ricordi che il JDK fornisce un metodo `Collections.sort()` per ordinare liste di oggetti confrontabili.

1. Si prenda in considerazione la seguente classe Java 8+, parametrica su due generics `X` e `Y`, le cui istanze rappresentano sequenze *iterabili* di coppie di coordinate cartesiane $X \times Y$ che rappresentano i punti individuati da una funzione $X \to Y$ in un determinato intervallo del dominio X.

```java
public class FunSeq<X extends Number & Comparable<? super X>, Y extends Number>
    implements Iterable<Pair<X, Y>> {

    private final X a, b;
    private final Function<? super X, ? extends Y> f;
    private final Function<X, X> inc;

    public FunSeq(X a, X b, Function<? super X, ? extends Y> f, Function<X, X> inc) {
        this.a = a;
        this.b = b;
        this.f = f;
        this.inc = inc;
    }

    /* da finire di implementare */
}
```

Al costruttore vengono passati l'intervallo di dominio $[a, b)$, la funzione f e la funzione *inc* per incrementare l'ascissa. La funzione f è la funzione principale: per ogni ascissa x di tipo `X` compresa nell'intervallo $[a, b)$, l'applicazione $f(x)$ ha tipo `Y`. La sequenza iterabile consiste in coppie $(x, f(x))$ di tipo `Pair<X, Y>` che rappresentano l'ascissa e l'ordinata di ogni punto. Essendo impossibile rappresentare funzioni continue, le ascisse procedono in maniera discreta secondo un valore di incremento: la funzione *inc* è necessaria proprio per calcolare la prossima ascissa nella sequenza.

(a) Si definisca una classe `Pair` parametrica su 2 generics `A` e `B` che rappresenta coppie *immutabili*.

(b) Si finisca di implementare la classe `FunSeq` opportunamente. Per realizzare il confronto delle ascisse, si noti che il type parameter `X` è vincolato non solo ad essere un `Number` ma anche a implementare `Comparable<X>`.

(c) Si implementi uno snippet di codice che utilizzi la classe `FunSeq` per rappresentare la parabola $x^2 + 2x - 1$ nel piano $\mathbb{R} \times \mathbb{R}$ nell'intervallo di dominio $[-2, 2)$ con incremento di ascissa 0.1. Ciò significa che le ascisse partono da -2 e procedono a step di 0.1 fino a 2 (escluso), producendo $((2 - 0.1) - (-2))/0.1 = 39$ punti. Si definisca la parabola con una opportuna lambda.

(d) Si implementi uno snippet simile a quello del punto precedente ma che rappresenti la parabola data nel piano $\mathbb{R} \times \mathbb{Z}$, ovvero con ordinate di tipo `int`.

2.1.8 Alberi binari di ricerca

[*Esame 01/06/2023*]

1. Vogliamo realizzare in Java 8+ una classe `BST`, parametrica su un tipo generico `T`, che rappresenta alberi binari di ricerca (Binary Search Tree) i cui nodi sono decorati con valori di tipo `T`. Un albero binario di ricerca è un normale albero binario, tuttavia gli elementi al suo interno vengono mantenuti con un certo ordine dato da una peculiare proprietà ricorsiva. Si prenda in esame il seguente esempio che per semplicità utilizza degli interi:

```
      11
    /    \
   6      25
  / \       \
 3   8      62
           /  \
         57    81
```

Il nodo radice 11 ha un valore superiore a tutti i nodi del sotto-albero sinistro ed inferiore a tutti i nodi del sotto-albero destro. E questo è vero non solo prendendo in esame la radice dell'albero, ma prendendo qualsiasi nodo in qualunque punto dell'albero!

Quando si inserisce un nuovo nodo nell'albero è necessario metterlo nel punto giusto, rispettando questa proprietà, così che l'albero sia sempre in uno stato valido.

Segue l'implementazione da completare della classe `BST`.

```java
public class BST<T> implements Iterable<T> {
    protected final Comparator<? super T> cmp;
    protected Node root;

    protected class Node {
        protected final T data;
        protected Node left, right;

        protected Node(T data, Node left, Node right) {
            this.data = data;
            this.left = left;
            this.right = right;
        }
    }

    public BST(Comparator<? super T> cmp) {
        this.cmp = cmp;
    }

    public void insert(T x) {
        root = insertRec(root, x);
    }

    protected Node insertRec(Node n, T x) { /* da implementare */ }

    protected void dfsInOrder(Node n, Collection<T> out) { /* da implementare */ }

    @Override
    public Iterator<T> iterator() { /* da implementare */ }

    public T min() { /* da implementare */ }
    public T max() { /* da implementare */ }
}
```

(a) Si implementi ricorsivamente il metodo `insertRec()` badando a rispettare la proprietà degli alberi binari di ricerca enunciata sopra. Per determinare se discendere nel sotto-ramo sinistro oppure in quello destro è necessario utilizzare il `Comparator` passato in costruzione per confrontare l'argomento `x` con il campo `data` del nodo `n`.

2.1.9 Alberi binari

[*Esame 13/09/2022 – 20/06/2019*]

1. Vogliamo realizzare in Java 8+ una classe `TreeNode`, parametrica su un tipo generico `T`, che rappresenta nodi di un albero binario decorati con valori di tipo `T`. Quando entrambi i sotto-alberi sinistro e destro di un nodo sono assenti, allora il nodo rappresenta una foglia. Seguono ora le specifiche dettagliate.

   (a) Gli alberi devono essere *iterabili* e l'iteratore deve attraversare l'albero in DFS (*Depth-First Search*), fornendo gli elementi di tipo `T` in *pre-ordine*, ovvero nell'ordine mostrato da questo esempio:

```
      1
    /   \
   2     5
  / \     \
 3   4     6
            / \
           7   8
```

   (b) Gli alberi devono essere confrontabili tramite il metodo `equals()`. Due alberi sono uguali se tutti i sotto-alberi sono uguali e sono costituiti da elementi uguali.

(c) Si definiscano gli opportuni costruttori ed eventualmente dei metodi statici che fungono da pseudo-costruttori. L'obiettivo è fare in modo che gli alberi siano facili da costruire innestando ricorsivamente le chiamate. Non si permetta l'istanziazione di alberi vuoti o non inizializzati da popolare successivamente con dei setter.

(d) Si implementino uno o più snippet di test che mettono alla prova tutte le caratteristiche richieste.

(e) Si implementi un *pretty printer* tramite un override del metodo `toString()`.

Si implementi la classe `TreeNode` secondo le specifiche date, rappresentando la struttura dati nella maniera che si ritiene più conveniente e fornendo tutti i metodi necessari. È importante il riuso di codice, l'information hiding ed una implementazione che escluda il più possibile stati di invalidità grazie ad un saggio uso dei tipi.

2.1.10 SkippableArrayList

[*Esame* ***01/07/2022*** *– 08/09/2018*] Si implementi in Java 8+ una sottoclasse generica di `java.util.ArrayList` di nome `SkippableArrayList` che estende la superclasse con un iteratore in grado di discriminare gli elementi secondo un predicato booleano. Gli elementi che soddisfano il predicato vengono processati da una certa funzione di trasformazione[5]; gli altri vengono passati ad una seconda callback (non una funzione di trasformazione).

1. Realizziamo in Java 8+ una sottoclasse di `java.util.ArrayList` di nome `SkippableArrayList` parametrica su un tipo `T` che estende la superclasse con un iteratore in grado di discriminare gli elementi secondo un predicato e di processarli tramite due funzioni distinte a seconda dell'esito dell'applicazione del predicato all'elemento.

   (a) Si definisca una *interfaccia funzionale* di nome `Predicate` specializzando l'interfaccia generica `java.util.Function` del JDK in modo che il dominio sia un generic `T` ed il codominio sia `Boolean`.

   (b) Si definisca una interfaccia `Either` parametrica su un tipo generico `T` e che definisce due metodi. Il primo metodo, di nome `onSuccess`, prende un `T` e ritorna un `T` e viene chiamato dall'iteratore quando il predicato ha successo. Il secondo metodo, di nome `onFailure`, viene invocato invece quando il predicato fallisce, prende un argomento di tipo `T` e non produce alcun risultato, tuttavia può lanciare una eccezione di tipo `Exception`:

   (c) Si definisca la sottoclasse `SkippableArrayList` parametrica su un tipo `E` e si implementi un metodo pubblico avente firma `Iterator<E> iterator(Predicate<E> p, Either<E> f)` che crea un iteratore con le caratteristiche accennate sopra. Più precisamente:

   - l'iteratore parte sempre dall'inizio della collezione ed arriva alla fine, andando avanti di un elemento alla volta normalmente;
   - ad ogni passo l'iteratore applica il predicato `p` all'elemento di tipo `T` corrente, che chiameremo `x`: se `p(x)` computa `true` allora viene invocato il metodo `onSuccess` di `f` e passato l'elemento `x` come argomento; altrimenti viene invocato il metodo `onFailure` e passato `x` come argomento a quest'ultimo;
   - l'invocazione di `onFailure` deve essere racchiusa dentro un blocco che assicura il *trapping* delle eccezioni: in altre parole, una eccezione proveniente dall'invocazione di `onFailure` non deve interrompere l'iteratore;
   - quando viene invocato `onSuccess`, il suo risultato viene restituito come elemento corrente dall'iteratore;
   - quando viene invocato `onFailure`, l'iteratore ritorna l'elemento originale che ha fatto fallire il predicato.

   (d) Si scriva un esempio di codice `main` che:

   - costruisce una `ArrayList` di interi vuota di nome `dst`;
   - costruisce una `SkippableArrayList` di interi di nome `src` e la popola con numeri casuali compresi tra 0 e 10, inclusi gli estremi[6];
   - invocando **solamente una volta** il metodo `iterator(Predicate<E>, Either<E>)` di `src` con gli argomenti opportuni, somma 1 a tutti gli elementi di `src` maggiori di 5 e appende in coda a `dst` quelli minori o uguali a 5.
   - Il metodo `iterator()` con due parametri richiesto dal punto (c) è un override o un overload?

2.1.11 Figure geometriche

[*Esame* ***03/06/2022*** *– 31/01/2020 – 24/05/2018*] Definiamo in Java 8+ un sistema di classi ed interfacce che rappresentano figure geometriche piane e solide. Le figure geometriche rappresentate non sono posizionate nel piano cartesiano o nello spazio, sono pertanto prive di coordinate. Per semplicità esse contengono solamente le informazioni sulla lunghezza dei lati o delle facce di cui sono costituite.

[5] Una funzione di trasformazione è una funzione in cui dominio è uguale al codominio, per esempio una funzione $f : \tau \to \tau$ è una funzione di trasformazione sull'insieme τ.

[6] Si utilizzi la classe `Random` del JDK per generare numeri casuali.

1. Prima di cominciare, realizziamo una piccola libreria interna che consiste in alcuni metodi statici generici altamente riusabili. Sia data la funzione di ordine superiore `fold` implementata tramite un metodo statico pubblico:

```
public static <T, State> State fold(Iterable<T> i, final State st0, BiFunction<State, T,
↪ State> f) {
    State st = st0;
    for (final T e : i)
    st = f.apply(st, e);
    return st;
}
```

   (a) Si implementi tramite **una sola** invocazione di `fold()` la funzione di ordine superiore `sumBy` avente la seguente firma:

```
public static <T> double sumBy(Iterable<T> i, Function<T, Double> f);
```

   Essa calcola la sommatoria di tutti gli elementi di i trasformandoli in `double` tramite `f`. La utilizzeremo ad esempio per calcolare il perimetro di un poligono sommando la lunghezza di tutti i suoi lati, oppure l'area laterale totale di un solido sommando l'area di tutte le superfici piane di cui è costituito.

   (b) Avremo bisogno di ordinare le nostre figure geometriche sulla base di diversi criteri, ad esempio l'area o il volume. Si implementi il metodo statico `compareBy()` avente la seguente firma:

```
public static <T> int compareBy(T s1, T s2, Function<T, Double> f);
```

   Esso confronta `s1` ed `s2` conventerdoli prima in `double` tramite `f`, riducendo pertanto il confronto al confronto tra due numeri `double`.

   (c) Quale forma di polimorfismo forniscono i generics definiti sulla firma di un metodo come ad esempio quelli del metodo `fold()` di cui sopra?

   ◯ Polimorfismo subtype ◯ Polimorfismo parametrico ◯ Polimorfismo parametrico first-class
   ◯ Polimorfismo ad-hoc

2. Definiamo ora i tipi essenziali per rappresentare figure geometriche piane e solide.

   (a) La classe `Edge` rappresenta grandezze 1-dimensionali come lati di poligoni, spigoli di poliedri, segmenti e circonferenze.

```
public class Edge implements Comparable<Edge> {
    private final double len;
    public Edge(double len) { this.len = len; }
    public double length() { return len; }
    @Override
    public int compareTo(Edge s) { /* DA IMPLEMENTARE */ }
    }
}
```

   Si implementi il metodo `compareTo()` tramite **una sola** invocazione della `compareBy()` definita sopra.

   (b) L'interfaccia `Surface` rappresenta figure piane qualunque:

```
public interface Surface extends Comparable<Surface> {
    double area();
    double perimiter();
    @Override
    default int compareTo(Surface s) { /* DA IMPLEMENTARE */ }
}
```

   Si dia una implementazione di default del metodo `compareTo()` tramite **una sola** invocazione della `compareBy()` definita sopra che esegua il confronto tra le aree.

   (c) Il sotto-tipo `Polygon` rappresenta poligoni: l'interfaccia specializza `Surface` dando una implementazione di default al metodo `perimeter()` e permette anche l'iterazione dei lati di cui il poligono stesso è costituito.

```
public interface Polygon extends Surface, Iterable<Edge> {
    @Override
    default double perimiter() { /* DA IMPLEMENTARE */ }
}
```

Si dia una implementazione di default del metodo `perimeter()` tramite **una sola** invocazione della `sumBy()` definita sopra.

(d) L'interfaccia `Solid` rappresenta solidi qualunque:

```
public interface Solid extends Comparable<Solid> {
    double outerArea(); // area laterale totale
    double volume();
    @Override
    default int compareTo(Solid s) { /* DA IMPLEMENTARE */ }
}
```

Si implementi il metodo `compareTo()` tramite **una sola** invocazione della `compareBy()` definita sopra che esegua il confronto tra i volumi.

(e) `Polyhedron` è un sottotipo di `Solid` e rappresenta poliedri. L'interfaccia è parametrica rispetto al sottotipo di `Polygon` che descrive le facce di cui il poliedro è costituito. Ad esempio, le facce di un cubo sono quadrati: una classe `Cube` implementerebbe pertanto l'interfaccia `Polyhedron` avente `Square` come *type argument*, `Square` $\preceq$ `Polygon`.

Un `Polyhedron` permette l'iterazione delle facce poligonali di cui esso è costituito; fornisce inoltre una implementazione di default del metodo `outerArea()` che calcola semplicemente la sommatoria delle aree delle sue facce.

```
public interface Polyhedron<P extends Polygon> extends Solid, Iterable<P> {
    @Override
    default double outerArea() { /* DA IMPLEMENTARE */ }
}
```

Si implementi il metodo `outerArea()` tramite **una sola** invocazione della `sumBy()` definita sopra.

3. Si proceda ora alla definizione di una gerarchia di classi che rappresentano figure geometriche specifiche implementando le interfacce fin qui introdotte.

   (a) Si implementi una classe che rappresenta *sfere* immutabili avente nome `Sphere` e che implementa l'interfaccia `Solid`. Il costruttore di `Sphere` deve prendere come parametro solamente un `double`: il raggio della sfera.

   (b) Si implementi una classe che rappresenta *cilindri* immutabili avente nome `Cilinder` e che implementa l'interfaccia `Solid`. Il costruttore di `Cilinder` deve prendere come parametri due `double`: il raggio della base e l'altezza del cilindro.

   (c) Si implementi una classe che rappresenta *rettangoli* immutabili avente nome `Rectangle` e che implementa l'interfaccia `Polygon`. Il costruttore di `Rectangle` deve prendere come parametri due `double`: base e altezza.

   (d) Si implementi una classe che rappresenta quadrati immutabili avente nome `Square` e che estende `Rectangle`. Il costruttore di `Square` deve prendere un solo parametro di tipo `double`: il lato.

4. Si prenda in considerazione la seguente classe che rappresenta parallelepipedi immutabili. I parallelepipedi sono poliedri aventi facce rettangolari.

```
public class Parallelepiped implements Polyhedron<Rectangle> {
    protected final double width, height, depth;
    public Parallelepiped(double width, double height, double depth) {
        this.width = width;
        this.height = height;
        this.depth = depth;
    }
    @Override
    public double volume() { /* DA IMPLEMENTARE */ }
    @Override
    public Iterator<Rectangle> iterator() {
        Rectangle r1 = new Rectangle(width, height),
                  r2 = new Rectangle(width, depth),
                  r3 = new Rectangle(height, depth);
        return List.of(r1, r2, r3, r1, r2, r3).iterator();
    }
}
```

(a) Si implementi il metodo `volume()`.

(b) Si definisca la classe `Cube` come sottoclasse di `Parallelepiped`. Il costruttore di `Cube` deve prendere solamente un parametro di tipo `double`: la lunghezza dello spigolo.

(c) È possibile co-variare il tipo di ritorno del metodo `iterator()` di `Cube` in modo che ritorni un `Iterator<Square>`? Si motivi la risposta.

(d) Assumendo l'implementazione di `Cube` di cui al punto (b), si prenda in considerazione il seguente snippet:

```
int facet_cnt = 1;
for (Square sq : new Cube(10.)) {
    int side_cnt = 1;
    for (Edge e : sq) {
        System.out.printf("side #%d/%d = %f\n", side_cnt++, facet_cnt, e.length());
    }
    ++facet_cnt;
}
```

Tale codice compila? Si motivi la risposta.

5. Si prenda in considerazione il seguente snippet di codice che crea una lista eterogena di poliedri le cui facce sono almeno rettangoli.

```
Cube c1 = new Cube(1.), c2 = new Cube(2.));
Parallelepiped p1 = new Parallelepiped(1., 2., 3.), p2 = new Parallelepiped(2., 3., 4.);
List<Polyhedron<? extends Rectangle>> polys = List.of(c1, c2, p1, p2);
```

Seguono ora alcune chiamate ai metodi `Collections.sort()` del JDK. Gli oggetti nella lista `polys` sono sempre gli stessi 4 (i cubi `c1` e `c2` ed i parallelepipedi `p1` e `p2`), ma ogni invocazione di `sort()` li ordina in maniera diversa a seconda del criterio di ordinamento. Per ciascuna invocazione si indichi quale oggetto viene messo in testa alla lista, ovvero quale oggetto computerebbe l'espressione `polys.get(0)`, oppure se non compila.

```
Collections.sort(polys)
```

◯ non compila ◯ c1 ◯ c2 ◯ p1 ◯ p2

```
Collections.sort(polys, (x, y) -> compareBy(x, y, Polyhedron::outerArea))
```

◯ non compila ◯ c1 ◯ c2 ◯ p1 ◯ p2

```
Collections.sort(polys, (x, y) -> compareBy(x, y, (p) -> p.outerArea()))
```

◯ non compila ◯ c1 ◯ c2 ◯ p1 ◯ p2

```
Collections.sort(polys, (x, y) -> compareBy(x, y, (r) -> r.perimiter()))
```

◯ non compila ◯ c1 ◯ c2 ◯ p1 ◯ p2

```
Collections.sort(polys, (x, y) -> compareBy(x, y, new Function<>() {
    public Double apply(Polyhedron<? extends Rectangle> r) {
        return r.volume();
}
}))
```

◯ non compila ◯ c1 ◯ c2 ◯ p1 ◯ p2

```
Collections.sort(polys, (x, y) -> Double.compare(sumBy(x, Square::perimiter),
                                                 sumBy(y, Rectangle::perimiter)))
```

◯ non compila ◯ c1 ◯ c2 ◯ p1 ◯ p2

2.1.12 Artisti

[*Esame 20/06/2019*] Si consideri la seguente interfaccia parametrica in linguaggio Java:

```java
public interface Equatable<T> {
    boolean equalsTo(T x);
}
```

Essa non sostituisce il meccanismo basato sul metodo `equals(Object)` della classe `Object`, tuttavia permette di implementare il confronto di uguaglianza in maniera più sicura delegando parte della logica ad un metodo fortemente tipato.

Si consideri ora la classe parametrica `Person`, che supporta il confronto di uguaglianza con oggetti di tipo `P`:

```java
public class Person<P extends Person<P>> implements Equatable<P> {
    public final String name;
    public final int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public boolean equals(Object o) { /* da implementare */ }

    @Override
    public boolean equalsTo(P other) { /* da implementare */ }

    @Override
    public String toString() { return name; }
}
```

1. Si implementi il metodo `equals(Object)` della classe `Person` in modo che, dati `p1` e `p2` di tipo `Person`, le seguenti invarianti siano rispettate:
   - `p1.equals(p1) == true`
   - `p1.equals(null) == false`
   - `p1.equals(e) == false` se l'espressione `e` ha tipo diverso dal tipo di `p1`[7]
   - `p1.equals(p2) == p1.equalsTo(p2)`
2. Si implementi il metodo `equalsTo(P)` della classe `Person` in modo che due oggetti siano uguali quando hanno il medesimo nome e la medesima età.

Si consideri ora il seguente codice:

```java
public class Artist extends Person<Artist> {
    public final Hair hair;

    public Artist(String name, int age, Hair hair) {
        super(name, age);
        this.hair = hair;
    }

    @Override
    public boolean equalsTo(Artist other) { /* da implementare */ }
}

public class Hair implements Equatable<Hair> {
```

[7] Ricordiamo che il metodo della classe `Object` avente firma `Class<? extends Object> getClass()` consente di estrarre a runtime il tipo *raw* di un oggetto.

```java
    public final int length;
    public final Set<Color> colors;

    public Hair(int length, Set<Color> colors) {
        this.colors = colors;
        this.length = length;
    }

    @Override
    public boolean equals(Object o) { /* da implementare */ }

    @Override
    public boolean equalsTo(Hair x) { /* da implementare */ }
}

public enum Color {
    BROWN, DARK, BLONDE, RED, GRAY;
}
```

1. Si implementino i metodi `equals(Object)` e `equalsTo(Hair)` della classe `Hair` in modo che due oggetti siano uguali quando hanno la medesima lunghezza ed i medesimi colori (indipendentemente dal loro ordine). Si rispettino le stesse invarianti del punto (a) per l'implementazione del metodo `equals(Object)` e si deleghi, come per la classe `Person`, la parte fortemente tipata del confronto al metodo `equalsTo(Hair)`.

2. Il metodo `equalsTo(Artist)` è davvero un *override* del metodo `equalsTo(P)` ereditato dalla superclasse?

   ◯ No, perché la *type erasure* elimina il generic `P` che viene sostuito col suo constraint `Person` nella superclasse, la quale introduce di fatto un metodo `equalsTo(Person)`, di cui `equalsTo(Artist)` non è un override ma un overload.

   ◯ Sì, perché il metodo `equalsTo()` viene gestito in modo particolare dal compilatore Java, permettendo override anche con firme differenti.

   ◯ No, perché la firma è diversa e quindi è un overload, non un override.

   ◯ Sì, perché il generic `P` viene sostituito col tipo `Artist` nello scope della sottoclasse, pertanto viene ereditato un metodo `equalsTo(Artist)`, rendendo questo un vero override.

3. Si implementi il metodo `equalsTo(Artist)` della classe `Artist` in modo che due oggetti siano uguali quando hanno i medesimi nome, età e capelli. Si badi a riusare opportunamente l'implementazione ereditata dalla superclasse.

   Si considerino ora i seguenti binding in Java:

```java
Person alice = new Person("Alice", 23),
       david = new Artist("Bowie", 69, new Hair(75, Set.of(Color.RED, Color.BROWN,
       ↪  Color.GRAY)));
Artist morgan = new Artist("Morgan", 47, new Hair(20, Set.of(Color.GRAY, Color.DARK))),
       madonna = new Artist("Madonna", 60, new Hair(50, Set.of(Color.BLONDE)));
```

4. Si prenda in considerazione il seguente metodo della classe `java.util.Collections` del JDK 7+ per computare l'elemento massimo in una collection secondo un criterio di ordinamento dato:

```java
static <T> T max(Collection<? extends T> c, Comparator<? super T> cmp)
```

   Si considerino ora i seguenti binding in Java:

```java
List<Artist> artists = Arrays.asList((Artist) david, morgan, madonna);
List<Person> persons = Arrays.asList(alice, david, morgan, madonna);
```

   (a) Si scriva uno statement di invocazione del metodo `max()` che computi l'oggetto della lista `artists` avente il più alto prodotto tra lunghezza dei capelli e numero di colori.

   (b) Si scriva uno statement di invocazione del metodo `max()` che computi l'oggetto della lista `persons` avente il nome che viene per primo in ordine lessicografico.

   (c) Il seguente statement sarebbe accettato dal compilatore Java 7+?

```java
Artist c = Collections.max(artists, new Comparator<Person>() {
    public int compare(Person a, Person b) {
        return a.age - b.age;
    }
});
```

2.1.13 Iteratori

[*Esame **30/05/2019** – 22/01/2019*] Si implementi il seguente sistema di iteratori e trasformazioni tra iteratori.

1. Si definisca una *interfaccia funzionale* simile a `java.util.BiFunction` che rappresenta funzioni binarie, parametrica sui 2 tipi degli argomenti e sul tipo di ritorno.

2. Si definisca un tipo per la coppia eterogenea immutabile, ovvero una classe `Pair` parametrica su 2 tipi distinti che rappresentano rispettivamente il tipo del primo e del secondo elemento della coppia.

3. Si definisca un tipo per la tripla eterogenea immutabile, ovvero una sottoclasse di `Pair` di nome `Triple`, parametrica su 3 tipi distinti che rappresentano i tipi dei 3 elementi.

4. Si definisca un metodo statico `evalIterator` generico su 3 tipi `A`, `B` e `C` che, dato un iteratore su coppie $\mathtt{A} * \mathtt{B}$ ed una funzione binaria $\mathtt{A} * \mathtt{B} \rightarrow \mathtt{C}$, produce un nuovo iteratore su triple $\mathtt{A} * \mathtt{B} * \mathtt{C}$ che si comporta come un *wrapper* di quello in input. L'iteratore in output applica la funzione binaria ad ogni coppia di elementi letti dall'iteratore in input e produce una tripla con i due valori appena letti e passati alla funzione assieme al risultato di quest'ultima. Si utilizzino i tipi `Pair`, `Triple` e `BiFunction` definiti nei punti precedenti.

2.1.14 Sequenza di Fibonacci

[*Esame 30/05/2019*]

1. Si implementi un classe `FiboSequence` le cui istanze rappresentano sequenze contigue di numeri di Fibonacci di lunghezza data in costruzione. Tali istanze devono essere *iterabili* tramite il costrutto *for-each* di Java, devono pertanto implementare l'interfaccia parametrica del JDK `java.util.Iterable<T>`. Ad esempio, il seguente codice deve compilare e stampare i primi 100 numeri di Fibonacci:

```java
for (int n : new FiboSequence(100)) {
    System.out.println(n);
}
```

   Requisito obbligatorio è che i numeri di Fibonacci generati dall'iteratore[8] sottostiano ad un meccanismo di *caching* che ne allevia il costo computazionale memorizzando il risultato di ogni passo di ricorsione, in modo che ogni computazione successiva con il medesimo input costi solamente un accesso in lettura alla cache.

2. Si scriva il codice *de-zuccherato* dello statement *for-each* di cui al punto precedente.

3. Si riscriva la classe `FiboSequence` in modo che la cache sia condivisa tra molteplici istanze.

2.1.15 Classe Random

[*Esame 04/09/2018*] Si prenda in considerazione questa semplificazione della classe `java.util.Random` del JDK: essenzialmente essa offre un costruttore senza parametri, un secondo costruttore con il seed per inizializzare il PRNG ed alcuni metodi per generare valori numerici di tipo differente:

```java
public class Random {
    public Random() { ... }
    public Random(int seed) { ... }
    public boolean nextBoolean() { ... }
    public int nextInt() { ... }
    public double nextDouble() { ... }
}
```

[8] Nell'esempio l'iteratore è implicitamente utilizzato dal costrutto *for-each*.

1. Si scriva un *wrapper* della classe `Random` che si comporta come un **singleton**, facendo attenzione a riprodurre ogni aspetto dell'originale.

2. Si definisca una classe `RandomIterator` che implementa l'interfaccia `java.util.Iterator<Integer>` del JDK e che si comporta come un iteratore su interi, generando un numero casuale ad ogni invocazione del metodo `next()` anziché scorrendo una vera collection, fino ad esaurire la sequenza di lunghezza specificata in costruzione. Si implementino opportunamente il costruttore ed i metodi richiesti dall'interfaccia.

2.1.16 FancyArrayList

[*Esame 20/06/2018*] Si implementi in Java una sottoclasse generica di `ArrayList` di nome `FancyArrayList` che estende le funzionalità della superclasse con un iteratore più versatile. Il nuovo iteratore deve essere in grado di andare sia avanti che indietro secondo un valore di incremento intero non nullo; e di processare gli elementi che incontra durante l'attraversamento applicando una funzione di trasformazione[9].

1. Si definisca una *interfaccia funzionale* di nome `Function` parametrica sia sul tipo del dominio che sul tipo del codominio, equivalente a quella definita dal JDK 8+ nel package `java.util.function`.

2. Si definisca la sottoclasse `FancyArrayList` parametrica su un tipo `E` e si implementi un metodo pubblico avente firma `Iterator<E> iterator(int step, Function<E, E> f)` che crea un iteratore con le caratteristiche accennate sopra tramite una *classe anonima*. Più precisamente:

   - quando il valore del parametro `step` è positivo, l'iteratore parte dall'inizio della collezione e va *avanti* incrementando il cursore di `step` posizioni ad ogni passo; quando invece `step` è negativo, l'iteratore parte dalla fine della collezione e va *indietro* decrementando il cursore;
   - ad ogni passo l'iteratore applica la funzione di trasformazione `f` all'elemento da restituire.

3. Si aggiungano a `FancyArrayList` i seguenti metodi pubblici, badando ad implementarli in funzione del metodo `iterator(int, Function<E, E>)` realizzato per l'esercizio precedente, *senza replicazioni* di codice. Per ciascuno si specifichi inoltre se è un override oppure no, utilizzando opportunamente l'annotazione `@Override`.
   (a) Si implementi il metodo avente firma `Iterator<E> iterator()` che produce un iteratore convenzionale che procede in avanti di una posizione alla volta.
   (b) Si implementi il metodo avente firma `Iterator<E> backwardIterator()` che produce un iteratore rovescio che procede indietro di una posizione alla volta.

4. Si rifattorizzi il metodo `iterator(int, Function<E, E>)` realizzato per il punto (2) in modo che non usi una classe anonima, ma una nuova classe innestata *statica* e parametrica di nome `FancyIterator`. Si presti particolare attenzione all'uso dei generics ed al passaggio esplicito della *enclosing instance* al costruttore.

2.1.17 Minimo e massimo in una lista

[*Esame 20/06/2018*] Si realizzi in Java un algoritmo che trova il minimo ed il massimo in una lista generica e restituisca i risultati, rispettivamente, come primo e secondo elemento di una coppia omogenea.

1. Si definisca un tipo per la coppia eterogenea, ovvero una classe `Pair` parametrica su due tipi distinti `A` e `B` che rappresentano rispettivamente il tipo del primo e del secondo elemento della coppia.

2. Si implementi un metodo statico e generico avente la seguente firma[10]:

   ```
   static <E> Pair<E, E> findMinAndMax(List<E> l, Comparator<E> c)
   ```

   L'algoritmo di ricerca del minimo e del massimo deve eseguire *una sola traversata* della lista; si assuma che gli argomenti siano non-nulli e che la lista abbia sempre almeno un elemento.

3. Si definisca un metodo in overload con il precedente che non usi un `Comparator` ma aggiunga il constraint `Comparable` al generic `E`, secondo la seguente firma:

   ```
   static <E extends Comparable<E>> Pair<E, E> findMinAndMax(List<E> l)
   ```

   Si implementi questo metodo in funzione del precedente, *senza replicare* l'algoritmo di ricerca.

[9] Una funzione di trasformazione è una funzione in cui dominio è uguale al codominio, per esempio $f : \tau \to \tau$ per un qualche insieme τ.
[10] L'interfaccia parametrica `Comparator<T>` include un metodo binario avente firma `int compare(T a, T b)`, il quale confronta i due argomenti e ritorna un intero secondo la semantica standard del confronto *a tre vie* in Java.

2.1.18 Hanna Barbera

[*Esame 24/05/2018*] Si assumano le seguenti classi ed interfacce Java, innestate per brevità in una sola classe a top-level:

```java
public class HannaBarbera {

    public interface Food {
        int getWeight();
    }

    public interface Animal extends Food {
        void eat(Food f);
    }

    public static class Mouse implements Animal {
        private int weight;

        public Mouse(int weight) { this.weight = weight; }

        @Override
        public void eat(Food f) { weight += f.getWeight() / 2; }

        @Override
        public int getWeight() { return weight; }
    }

    public static class Cat implements Animal {
        private int weight;

        public Cat(int weight) { this.weight = weight; }

        @Override
        public void eat(Food f) { weight += f.getWeight() / 5; }

        @Override
        public int getWeight() { return weight; }
    }

    public static class Cheese implements Food {
        @Override
        public int getWeight() { return 50; }
    }
}
```

1. Come sarebbe possibile rifattorizzare questo codice in modo da ridurre al minimo le ripetizioni e massimizzare il riuso? Si scriva il codice rifattorizzato, facendo attenzione a non cambiare il comportamento del programma.

2. Si prenda in considerazione il seguente metodo `main()` per la classe `HannaBarbera`, supponendo che le restanti classi ed interfacce innestate siano quelle rifattorizzate secondo quanto richiesto dall'esercizio 1.

```java
public static void main(String[] args) {
    Animal tom = new Cat(200);
    Animal jerry = new Mouse(10);
    jerry.eat(new Cheese());
    jerry.eat(new Food() {
        @Override
        public int getWeight() { return 10; }
    });
```

```
    tom.eat(jerry);
    System.out.println(String.format("Tom now weights %d", tom.getWeight()));
}
```

(a) Qualora una o più parti del metodo `main()` non siano compatibili a causa della rifattorizzazione eseguita precedentemente, si scriva in maniera chiara quali modifiche sono necessarie per renderlo compilabile mantenendo la semantica dell'originale; se preferibile, si riscriva per chiarezza l'intero codice `main()` opportunamente modificato.

(b) Qual'è il peso del gatto Tom che viene scritto in output alla fine?

2.2 C++

2.2.1 Pair

[*Esame 10/09/2024 – **01/06/2023***]

1. Vogliamo implementare in C++ una classe pair templatizzata su due tipi generici `A` e `B` che rappresentano i tipi del primo e del secondo elemento della coppia. Si utilizzi la versione del linguaggio che si preferisce, è sufficiente dichiarare quale.

   (a) La classe `pair` deve rispettare gli stili della generic programming e comportarsi come un valore: si definiscano gli opportuni costruttori, tra cui quello per copia, e l'operatore di assegnamento.

   (b) Si implementino tutti i metodi e gli operatori ritenuti necessari o interessanti.

   (c) Si implementi un costruttore per copia templatizzato su due tipi generici `C` e `D` che è in grado di costruire una `pair<A,B>` dato un argomento di tipo `const pair<C, D>&`.

2.2.2 Matrici

[*Esame 25/06/2024 – 05/09/2023 – **03/06/2022***]

1. Si scriva in C++ una classe generica `matrix` che rappresenta matrici bidimensionali di valori di tipo `T`, dove `T` è un tipo templatizzato. Tale classe deve comportarsi come un *valore*, secondo lo stile dei *container* di STL, pertanto deve supportare il costruttore per copia, l'operatore di assegnamento ed il costruttore di default. Si aggiungano poi metodi e operatori a piacere, tra cui **almeno** i seguenti:

   - costruttore con 2 parametri + 1 opzionale: numero di righe, numero di colonne e valore iniziale di tipo `T`
   - distruttore (se necessario);
   - member type `iterator`, `const_iterator` e `value_type`;
   - accesso tramite riga e colonna implementando 2 overload (const e non-const) di `operator()`;
   - 2 overload (const e non-const) dei metodi `begin()` ed `end()` per poter iterare tutta la matrice dall'angolo superiore sinistro all'angolo inferiore destro come se fosse piatta.

   L'implementazione può utilizzare STL liberamente, se lo si desidera.

   Si prenda a riferimento il seguente snippet per la specifica dei requisiti del tipo `matrix`:

```
matrix<double> m1;          // non inizializzata
matrix<double> m2(10, 20);  // 10*20 inizializzata col default constructor di double
matrix<double> m3(m2);      // costruita per copia
m1 = m2;                    // assegnamento
m3(3, 1) = 11.23;           // operatore di accesso come left-value
for (typename matrix<double>::iterator it = m1.begin(); it != m1.end(); ++it) {
    typename matrix<double>::value_type& x = *it; // de-reference non-const
    x = m2(0, 2); // operatore di accesso come right-value
}
matrix<string> ms(5, 4, "ciao"); // 5*4 inizializzata col la stringa passata come terzo
↪ argomento
for (typename matrix<string>::const_iterator it = ms.begin(); it != ms.end(); ++it)
    cout << *it; // de-reference const
```

2.2.3 Sommatoria

[*Esame 14/06/2024*]

1. Si svolgano i serguenti esercizi in linguaggio C++[11].
   (a) Si implementi una funzione templatizzata su un tipo `C` che, dato un container STL avente tipo `C`, ne somma tutti gli elementi tramite l'operatore di addizione e ritorna il risultato della sommatoria.
   (b) Quali vincoli sul template parameter `C` sono implicitamente richiesti? Si scrivano dettagliatamente tutti i vincoli per C e per gli eventuali member type utilizzati nell'implementazione.
   Esempio: il tipo `C` deve avere il tal metodo, deve supportare il tal costruttore, il tal operatore ecc.; poi deve avere un certo member type e quest'ultimo deve supportare il tal metodo, operatore ecc.

2.2.4 Map

[*Esame 10/01/2024*]

1. Si implementino in C++11 alcune varianti della funzione `map()` rispettando la sua semantica tipica: la funzione passata come argomento alla `map()` deve essere applicata ad ogni elemento della sequenza di input, producendo una sequenza di output con tutti i risultati delle applicazioni.
   (a) Si implementi la seguente versione della `map()` che opera su iteratori. Si badi che in questo caso la funzione non ha tipo di ritorno. L'output consiste in un iteratore passato come argomento: lì la funzione dovrà scrivere i risultati.

```
template <class InputIterator, class OutputIterator>
void map(InputIterator from, InputIterator to, OutputIterator out,
    function<typename OutputIterator::value_type(typename InputIterator::value_type)> f)
```

   (b) Si implementi la seguente versione della `map()` che opera su vector di STL. In questo caso l'output è un vero e proprio tipo di ritorno. L'implementazione deve invocare la `map()` di cui al punto precedente.

```
template <class A, class B>
vector<B> map(const vector<A>& v, function<B(A)> f)
```

   (c) Il tipo templatizzato `function` è definito da STL a partire dallo standard C++11, così come le espressioni lambda. Nessuno dei due esisteva in C++ *vanilla* (oggi chiamato C++03).
      i. Come sarebbe stato possibile scrivere in C++03 le firme delle `map()` di cui ai punti (a) e (b) senza usare il tipo `function`?
      ii. Le firme compatibili con C++03 potrebbero coesistere con le firme originali di cui ai punti (a) e (b)? In altre parole, sarebbero considerati tutti overload validi dal compilatore C++11?
      iii. Come sarebbe stato possibile invocare le `map()` compatibili con C++03 senza le lambda?

2.2.5 Funzioni

[*Esame 30/06/2023*]

1. Si implementi in C++ una classe templatizzata `fun_seq` analoga a quella dell'esercizio 2.1.7 (Java), adottando però i pattern, gli stili e le convenzioni di C++ e di STL. Alcuni suggerimenti:
   - si usi il tipo `std::pair` definito da STL per rappresentare le coppie;
   - l'iterabilità in C++ è rappresentata del *concept* denominato `Container` in STL;
   - la funzione di incremento passata al costruttore in Java non è necessaria in C++ e può essere sostituita dagli opportuni operatori;
   - il vincolo `Comparable` sul type parameter `X` in Java non ha equivalente in C++, è sufficiente assumere che il relativo template argument supporti un operatore di confronto.

   Si scriva anche uno snippet analogo a quello del punto c (Es. 2.1.7 Java).

[11] Non è necessaria nessuna revisione recente del linguaggio, l'esercizio è esprimibile in C++ vanilla, detto C++03.

2.2.6 Curve

[*Esame 13/01/2023*]

1. Si prenda in considerazione la seguente classe C++14 che rappresenta curve definite tramite funzioni unarie $\mathbb{R} \to \mathbb{R}$ in un certo intervallo di dominio $[a, b] \in \mathbb{R}$.

```cpp
#include <functional>
#include <iostream>
#include <utility>

using namespace std;
using real = double;
using unary_fun = function<real(const real&)>;

#define RESOLUTION (1000)   // risoluzione dell'intervallo [a, b]

class curve
{
private:
    real a, b;
    unary_fun f;
public:
    curve(const real& a_, const real& b_, const unary_fun& f_) : f(f_), a(a_), b(b_) {}
    curve(const real& c) = default;

    real get_dx() const { return (b - a) / RESOLUTION; }
    pair<real, real> interval() const { return pair<real, real>(a, b); }
    real operator()(const real& x) const { return f(x); }

    curve derivative() const { /* DA IMPLEMENTARE */ }
    curve primitive() const { /* DA IMPLEMENTARE */ }
    real integral() const { /* DA IMPLEMENTARE */ }

    class iterator
    {
    private:
        const curve& c;
        real x;
    public:
        iterator(const curve& c_, const real& x_) : c(c_), x(x_) {}
        iterator(const iterator& c) = default;

        pair<real, real> operator*() const { /* DA IMPLEMENTARE */ }
        iterator operator++() { /* DA IMPLEMENTARE */ }
        iterator operator++(int) { /* DA IMPLEMENTARE */ }
        bool operator!=(const iterator& it) const { /* DA IMPLEMENTARE */ }
    };

    iterator begin() const { /* DA IMPLEMENTARE */ }
    iterator end() const { /* DA IMPLEMENTARE */ }
};
```

   (a) Si implementi il metodo `derivative()` che calcola la derivata, cioè la curva con una lambda che calcola il rapporto differenziale $\frac{f(x+dx)-f(x)}{dx}$

   (b) Si implementi il metodo `primitive()` che calcola la primitiva, ovvero la curva che rappresenta l'integrale indefinito con una lambda che calcola il prodotto differenziale $f(x) \cdot dx$.

   (c) Si implementi il metodo `integral()` che calcola l'integrale definito $\int_a^b f(x)\ dx$, ovvero la sommatoria dei prodotti differenziali calcolati dalla primitiva nell'intervallo $[a, b]$.

(d) Si implementino i metodi relativi all'iteratore tenendo presente che:

- il metodo `begin()` computa un iteratore all'inizio dell'intervallo $[a, b]$;
- il metodo `end()` computa un iteratore oltre la fine dell'intervallo $[a, b]$, includendo l'estremo b nell'iterazione;
- l'operatore di de-reference dell'iteratore produce una coppia di reali che rappresentano un punto della curva sul piano cartesiano;
- gli operatori di pre e post incremento incrementano l'ascissa corrente x della quantità dx.

Il seguente snippet di test può servire da riferimento per capire il comportamento dell'iteratore:

```
curve c(-10., 10., [](const real& x) { return x * x - 2 * x + 1; });
for (curve::iterator it = c.begin(); it != c.end(); ++it)
{
    const pair<real, real>& p = *it;
    const real& x = p.first, & y = p.second;
    cout << "c(" << x << ") = " << y << endl;
}
```

2.2.7 Alberi binari

[*Esame 13/09/2022*]

1. Vogliamo realizzare in C++ lo stesso sistema di alberi binari di cui al Quesito 2.1.9 (Java). La traduzione del codice da Java a C++ non deve essere letterale: è necessario adottare gli stili e le convenzioni di C++ e di STL, ad esempio formulando il confronto tramite l'overloading degli opportuni operatori, definendo gli *opportuni member* type per gli iteratori e implementando gli overload const e non-const dei metodi laddove necessario. La classe `tre_node` deve avere un template parameter `T` ed aderire allo stile della *value*-oriented programming, comportandosi come un valore con gli opportuni costruttori e operatori di assegnamento. Segue la specifica dettagliata.

   (a) Gli alberi devono essere *iterabili* e l'iteratore deve attraversare l'albero in maniera algoritmicamente equivalente all'implementazione Java di cui al Quesito a (Es. 2.1.9 Java). Si definiscano i *member type* per iteratori const e non-const e le relative coppie di metodi `begin()` ed `end()`. Se necessario, implementare gli iteratori tramite classi ausiliarie.

   (b) La definizione di equivalenza di due alberi è semanticamente equivalente a quella definita nel Quesito b (Es. 2.1.9 Java). Si assuma che gli oggetti di tipo `T` siano confrontabili tramite l'operatore ==.

   (c) Si definiscano gli opportuni costruttori, operatori di assegnamento ed eventuali distruttori secondo lo stile della *value*-oriented programming. Se vantaggioso, si forniscano metodi statici che fungano da pseudo-costruttori e permettano di costruire alberi facilmente innestando ricorsivamente le chiamate.

   (d) Si implementino uno o più snippet di test.

   (e) Si implementi un *pretty printer* tramite un overload globale dell'operatore <<.

   Si utilizzi la revisione del linguaggio C++ che si preferisce, purché si specifichi quale. È importante fare buon uso del type system, riusando il codice laddove possibile.

2.2.8 Smart Pointer

[*Esame 01/07/2022*]

1. Si definisca in linguaggio C++ una classe `smart_ptr` templatizzata su un tipo `T` che implementi la logica di uno *smart pointer*. Si implementino tutti i costruttori, i distruttori e gli operatori che si ritengono necessari ed utili affinché uno smart pointer si comporti in maniera compatibile con un pointer C. In altre parole, uno smart pointer deve implementare non solo il *reference counting* ma deve anche comportarsi come un puntatore classico, inclusi gli operatori di incremento/decremento, l'aritmetica dei puntatori ed ovviamente il de-reference.

3 Domande aperte

Le seguenti domande servono a verificare le conoscenze dello studente relativamente agli argomenti coperti durante il corso. Lo studente deve dimostrare di aver studiato l'argomento ed essere in grado di esporlo in modo adeguato.

1. *Incapsulamento*: spiegare cosa si intende per metodo setter. Fornire almeno una motivazione per la quale l'uso di setter è preferibile alla definizione di campi pubblici. Fare un esempio concreto (con codice) di un setter che svolge una funzione non ottenibile con un campo pubblico.

2. *Ereditarietà*: spiegare approfonditamente in concetto di sovrascrittura di un metodo e quali sono le condizioni nelle quali può essere utile. Fare un esempio pratico mostrando le necessarie porzioni di codice.

3. *Espressioni Lambda*: considerare l'interfaccia `ActionListener` con l'unico metodo `void actionPerformed(ActionEvent e)`. Considerare il `JButton b` e mostrare (con codice) come utilizzare il metodo `void addActionListener(ActionListener l)` per aggiungere un `ActionListener` che stampi a console la stringa `"Java"`, codificato rispettivamente come una classe anonima e come un'espressione Lambda.

4 Soluzioni

4.1 Soluzione Tests

Sezione 1.1:

1. Singleton
2. Perché i costruttori non sono metodi soggetti al *dynamic dispatching*, quindi non è possibile sfruttare il polimorfismo per costruire oggetti se non tramite un metodo non-statico che incapsula la costruzione.

Sezione 1.2:

1. No, però non c'è limite al numero di interfacce che una classe può implementare.
2. A riusare lo stesso codice, ovvero chiamare un metodo, con parametri di tipo differente.
3. $\varphi \preceq \rho$
4. No, ma rende possibile il polimorfismo subtype poiché garantisce che i membri pubblici delle superclassi esistano nelle sottoclassi.
5. Il tipo ed il numero dei parametri e basta, senza tipo di ritorno né eccezioni.
6. Sì, anche se le sottoclassi `Paladin` e `Rogue` specializzano il tipo di ritorno rispetto al type argument passato alla superclasse, la specializzazione del tipo di ritorno è supportata dall'overriding in Java.
7. `static <A, B> Collection<B> map(Collection<A> c, Function<? super A, ? extends B> f)`
8. (a) `Paladin` il compilatore è in grado di discriminare.
   (b) Sì, compilerebbe e risolverebbe `normalizeAttack(Paladin)`.
9. L'overriding supporta la *co-varianza* del tipo di ritorno di un metodo.
10. No: l'overloading non è possibile tra metodi che differiscono solamente per il tipo di ritorno;

Sezione 1.3.1:

1. (a) false
   (b) true
   (c) non compila/eccezione[12]
   (d) false
   (e) false
   (f) true
   (g) false
   (h) false
   (i) non compila
   (j) false
   (k) false
   (l) non compila

Sezione 1.3.2:

1. (a) No, perché i metodi avrebbero la medesima *type erasure*.
   (b) No: il constraint `<T extends Comparable<T>>` impone che `Elf` implementi `Comparable<Elf>`, ma invece implementa `Comparable<Humanoid>`.
   (c) `b`
2. Sì: il primo argomento `artists` ha tipo `List<Artist>`, istanziando il generic `T` con `Artist`, pertanto l'oggetto di tipo `Comparator<Person>` passato come secondo argomento soddisfa il wildcard con *lower bound* perché `Person` è supertipo di `Artist`.

Sezione 1.3.3:

1. Ad un riferimento ad un metodo non-statico (*instance method reference*) di un oggetto specifico, che è `this` in questo caso; ovvero all'espressione `this::volume`.

[12] alcuni compilatori rifiutano `null.metodo()`, altri lo accettano ma a runtime viene lanciato `NullPointerException`

4.2 Soluzione esercizi di progettazione e implementazione - Java

4.2.1 Thread Pool (2.1.2)

```java

import java.util.Random;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;

public class Es1 {

    public interface Pool<R> {
        R acquire() throws InterruptedException;
        void release(R x);
    }

    // 2.b
    public static class SimplePool<T> implements Pool<T> {

        private final BlockingQueue<T> q = new LinkedBlockingQueue<>();

        @Override
        public T acquire() throws InterruptedException {
            return q.take();
        }

        @Override
        public void release(T x) {
            q.add(x);
        }
    }

    // 2.c + 2.d

    public interface Resource<T> {
        T get();
        void autorelease();
    }

    public static class AutoPool<T> implements Pool<Resource<T>> {

        private final BlockingQueue<T> q = new LinkedBlockingQueue<>();

        // questo metodo non è richiesto dall'interfaccia AutoPool, però è utile perché serve a popolare la pool, come accade nel main
        public void add(T x) {
            q.add(x);
        }

        public Resource<T> acquire() throws InterruptedException {
            T r = q.take();
            return new Resource<>() {

                @Override
                public T get() {
                    System.out.printf("acquired: %s%n", r);
                    return r;
                }
```

```java

                @Override
                public void autorelease() {
                    System.out.printf("released: %s%n", r);
                    add(r);
                }

                @SuppressWarnings("deprecation")
                @Override
                protected void finalize() {
                    autorelease();
                }
            };
        }

        @Override
        public void release(Resource<T> x) {
            x.autorelease();
        }
    }

    public static void main(String[] args) {
        AutoPool<Integer> pool = new AutoPool<>();
        Random rnd = new Random();
        for (int i = 0; i < 5; ++i) {
            pool.add(i);    // popolo la pool con alcuni oggetti
        }

        try {
            while (true) {
                Resource<Integer> r = pool.acquire();
                System.out.println("using " + r.get());
                Thread.sleep(Math.abs(rnd.nextInt() % 1000));
            }
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

    }


}

```

4.2.2 Piano Cartesiano (2.1.3)

```java
import java.util.*;

public class Es {

    // 1.a
    public static class Point {
        public final double x, y;

        public Point(double x, double y) {
            this.x = x;
            this.y = y;
        }
```

```java
    }

    // 1.b
    public static class Line {
        private final Point p1, p2;

        public Line(Point p1, Point p2) {
            this.p1 = p1;
            this.p2 = p2;
        }

        public double length() {
            return Math.sqrt(Math.pow(p1.x - p2.x, 2.) + Math.pow(p1.y - p2.y, 2.));
        }
    }

    // 1.c
    public abstract static class Polygon {
        protected final List<Point> points;

        protected Polygon(List<Point> points) {
            assert points.size() >= 3;  // 1.c.i: RISPOSTA: il motivo per cui è necessario questo assert è che non è possibile vincolare la lunghezza di una lista con il type system: i tipi non esprimono valori numerici statici
            this.points = points;
        }

        // 1.c.ii
        public Iterator<Line> lineIterator() {
            return new Iterator<>() {
                private final Iterator<Point> it = points.iterator();
                private Point last = it.next();     // ci sono sempre almeno 3 punti, non può fallire questa next()

                @Override
                public boolean hasNext() {
                    return it.hasNext();
                }

                @Override
                public Line next() {
                    Point p = it.next();
                    Line r = new Line(last, p);
                    last = p;
                    return r;
                }
            };
        }

        // 1.c.iii
        public double perimeter() {
            Iterator<Line> lineIt = lineIterator();
            double r = 0.;
            while (lineIt.hasNext()) {
                r += lineIt.next().length();
            }
            return r;
```

```java
        }

        public abstract double area();
    }

    // 2.a
    public static class Triangle extends Polygon {
        public Triangle(Point p1, Point p2, Point p3) {
            super(List.of(p1, p2, p3));
        }

        @Override
        public double area() {
            Point p1 = points.get(0), p2 = points.get(1);
            // versione facile con la base orizzontale (sufficiente per la valutazione dell'esame)
            if (p1.x == p2.x) {
                double base = new Line(p1, p2).length(), h = new Line(new Point(p2.x, p1.y), p2).length();
                return base * h / 2.;
            }
            // versione generale con la formula di Erone (non necessaria per la valutazione dell'esame)
            else {
                Point p3 = points.get(2);
                double p = perimeter() / 2.,    // semiperimetro
                        a = new Line(p1, p2).length(),
                        b = new Line(p2, p3).length(),
                        c = new Line(p3, p1).length();
                return Math.sqrt(p * (p - a) * (p - b) * (p - c));
            }
        }

    }

    // 2.b
    public static class RightTriangle extends Triangle {
        public RightTriangle(Point p1, double b, double h) {
            super(p1, new Point(p1.x + b, p1.y), new Point(p1.x, p1.y + h));
        }
    }

    // 2.c
    public static class Rectangle extends Polygon {
        public Rectangle(Point p1, Point p3) {
            super(List.of(p1, new Point(p1.x, p3.y), p3, new Point(p3.x, p1.y)));
        }

        @Override
        public double area() {
            Point p1 = points.get(0), p2 = points.get(1), p4 = points.get(3);
            Line b = new Line(p1, p4), h = new Line(p1, p2);
            return b.length() * h.length();
        }
    }

    // 2.d
```

```java
    public static class Square extends Rectangle {
        public Square(Point p1, double side) {
            super(p1, new Point(p1.x + side, p1.y + side));
        }

    }

    // 3.a
    static <T> T max(Collection<T> l, Comparator<? super T> cmp) {
        assert l.size() > 1;
        T max = l.iterator().next();    // prendo il primo elemento
        for (T x : l) {
            if (cmp.compare(x, max) > 0) max = x;
        }
        return max;
    }

    public static void main(String[] args) {
        Square sq1 = new Square(new Point(10., -4.), 0.1),
                sq2 = new Square(new Point(1., 20.), 0.01);
        Collection<Square> squares = List.of(sq1, sq2);
        Rectangle r = max(squares, new Comparator<Polygon>() {
            @Override
            public int compare(Polygon a, Polygon b) {
                return (int) (a.area() - b.area());
            }
        });
        // 3.b: il metodo max() ritorna sq2 perché la sottrazione nel confronto è invertita, quindi la ricerca del massimo in realtà restituisce il più piccolo anziché il più grande;
        //      ed il quadrato con area minore è sq2, perché 0.01^2 = 0.0001 mentre 0.1^2 = 0.01, quindi sq2 è di fatto il più piccolo.

        // 3.c: l'area di r (cioè di sq2) è 0.01^2 = 0.0001 = 10^-4
    }

}
```

4.2.3 Funzioni di ordine superiore (2.1.4)

```java
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.function.Consumer;
import java.util.function.Function;
import java.util.function.Supplier;

public class Es1 {

    public static <A, B> Iterator<B> mapIterator(Iterator<A> it, Function<A, B> f) {
        return new Iterator<>() {
            @Override
            public boolean hasNext() {
                return it.hasNext();
            }

            @Override
            public B next() {
```

```
                return f.apply(it.next());
            }
        };
    }

    public static <T> void forEach(Iterable<T> it, Consumer<T> f) {
        for (T x : it)
            f.accept(x);
    }

    public static class Pair<X, Y> {
        public final X fst;
        public final Y snd;
        public Pair(X x, Y y) {
            fst = x;
            snd = y;
        }
    }

    public static void main1(String[] args) {
        List<Pair<Function<String, Integer>, String>> l = new ArrayList<>();
        Iterator<Integer> it = mapIterator(l.iterator(), (p) -> p.fst.apply(p.snd));
    }

    public static <A, B> Iterator<B> applyFuns(Iterable<Pair<Function<A, B>, A>> l) {
        return mapIterator(l.iterator(), (p) -> p.fst.apply(p.snd));
    }

    public static void main2(String[] args) {
        List<Pair<Consumer<Integer>, Integer>> l = new ArrayList<>();
        forEach(l, (p) -> p.fst.accept(p.snd));
    }

    public static <A> void acceptFuns(Iterable<Pair<Consumer<A>, A>> l) {
        forEach(l, (p) -> p.fst.accept(p.snd));
    }

    public static <A, B> Iterator<Supplier<B>> asyncMapIterator(Iterator<A> it, Function<A, B>
    ↪ f) {
        return new Iterator<>() {
            @Override
            public boolean hasNext() {
                return it.hasNext();
            }

            @Override
            public Supplier<B> next() {
                final A x = it.next();
                final B[] r = (B[]) new Object[1];
                Thread t = new Thread(() -> { r[0] = f.apply(x); });
                t.start();
                return () -> {
                    try {
                        t.join();
                    } catch (InterruptedException e) {
                        throw new RuntimeException(e);
                    }
```

```
                    return r[0];
                };
            }
        };
    }

    public static <T> void asyncForEach(Iterable<T> it, Consumer<T> f) {
        for (T x : it)
            new Thread(() -> f.accept(x)).start();
    }
}
```

4.2.4 Fattoriale con thread (2.1.5)

```
import java.util.ArrayList;
import java.util.Collection;
import java.util.List;
import java.util.function.Function;

public class Es1 {

    // 1.a
    public static class FactorialThread extends Thread {
        private final int n;
        private long res;

        public FactorialThread(int n) {
            this.n = n;
        }

        @Override
        public void run() {
            res = fact(n);
        }

        public long getResult() {
            try {
                join();
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }
            return res;
        }

        public int getN() {
            return n;
        }

        private static long fact(int n) {
            if (n <= 1) return 1;
            return n * fact(n - 1);
        }

    }

    // 1.b + 1.c
    public static List<FactorialThread> parallelFactorial(Iterable<Integer> c) {
        List<FactorialThread> r = new ArrayList<>();
```

```java
        for (int n : c) {
            FactorialThread t = new FactorialThread(n);
            t.start();
            r.add(t);
        }
        return r;
    }

    // 1.d
    public static void main(String[] args) {
        for (FactorialThread t : parallelFactorial(List.of(0, 1, 2, 3, 11, 12, 23, 35)))   // chiamare anche parallelFactorial2() per provarla
            System.out.printf("fact(%d) = %d\n", t.getN(), t.getResult());
    }

    // 1.e.i
    public static <A, B> List<B> map(Iterable<A> i, Function<A, B> f) {
        List<B> r = new ArrayList<>();
        for (A a : i)
            r.add(f.apply(a));
        return r;
    }

    // 1.e.ii
    public static Collection<FactorialThread> parallelFactorial2(Collection<Integer> c) {
        return map(c, (n) -> { FactorialThread t = new FactorialThread(n); t.start(); return t; });
    }
}
```

4.2.5 Iteratori asincroni (2.1.6)

```java
import org.jetbrains.annotations.NotNull;
import org.jetbrains.annotations.Nullable;

import java.util.Collections;
import java.util.Iterator;
import java.util.List;
import java.util.function.Function;
import java.util.function.Supplier;

public class Main {

    // 1.a
    public static <A, B> Iterator<Supplier<B>> asyncIterator(Iterator<A> it, Function<A, B> f) {
        return new Iterator<>() {
            @Override
            public boolean hasNext() {
                return it.hasNext();
            }

            private class Future implements Supplier<B> {
                @Nullable
                private B r;
                @NotNull
                private final Thread t;

                public Future(Supplier<B> f) {
```

```
                    t = new Thread(() -> { r = f.get(); });
                    t.start();
                }

                @Override
                @NotNull
                public B get() {
                    try {
                        t.join();
                    } catch (InterruptedException e) {
                        throw new RuntimeException(e);
                    }
                    return r;
                }
            }

            @Override
            public Supplier<B> next() {
                A a = it.next();
                return new Future(() -> f.apply(a));
            }
        };
    }

    // 1.b
    static <T extends Comparable<? super T>> void sortLists(Iterable<List<T>> c) {
        Iterator<Supplier<List<T>>> it = asyncIterator(c.iterator(), (l) -> {
        ↪  Collections.sort(l); return l; });
        while (it.hasNext()) {
            Supplier<List<T>> f = it.next();
            System.out.println(f.get());    // questo non è davvero necessario, ma
        }
    }

}
```

4.2.6 Funzioni (2.1.7)

```
import java.util.Iterator;
import java.util.function.Function;

public class Es1 {

    // 1.a
    public static class Pair<A, B> {
        public final A fst;
        public final B snd;

        public Pair(A a, B b) {
            fst = a;
            snd = b;
        }
    }

    public static class FunSeq<X extends Number & Comparable<? super X>, Y extends Number>
        implements Iterable<Pair<X, Y>> {

        private final X a, b;
```

```java
        private final Function<? super X, ? extends Y> f;
        private final Function<X, X> inc;

        public FunSeq(X a, X b, Function<? super X, ? extends Y> f, Function<X, X> inc) {
            this.a = a;
            this.b = b;
            this.f = f;
            this.inc = inc;
        }

        // 1.b
        @Override
        public Iterator<Pair<X, Y>> iterator() {
            return new Iterator<>() {
                private X x = a;
                @Override
                public boolean hasNext() {
                    return x.compareTo(b) <= 0;
                }

                @Override
                public Pair<X, Y> next() {
                    Pair<X, Y> r = new Pair<>(x, f.apply(x));
                    x = inc.apply(x);
                    return r;
                }
            };
        }
    }

    // 1.c
    public static void test1() {
        for (Pair<Double, Double> p : new FunSeq<>(-2., 2., (x) -> x * x - 2 * x + 1, (x) -> x + 0.1)) {
            final double x = p.fst, y = p.snd;
            System.out.printf("f(%g) = %g\n", x, y);
        }
    }

    // 1.d
    public static void test2() {
        for (Pair<Double, Integer> p : new FunSeq<>(-2., 2., (x) -> (int) (x * x - 2 * x + 1), (x) -> x + 0.1)) {
            final double x = p.fst;
            final int y = p.snd;
            System.out.printf("f(%g) = %d\n", x, y);
        }
    }


    public static void main(String[] args) {
        test1();
        test2();
    }
}
```

4.2.7 Alberi binari di ricerca (2.1.8)

```java
import org.jetbrains.annotations.NotNull;
import org.jetbrains.annotations.Nullable;

import java.util.*;
import java.util.function.Function;
import java.util.function.Supplier;

public class Es1 {

    // NOTA: la classe è statica solamente perché è nested per praticità
    public static class BST<T> implements Iterable<T> {
        @NotNull
        protected final Comparator<? super T> cmp;
        @Nullable
        protected Node root;

        public BST(@NotNull Comparator<? super T> cmp) {
            this.cmp = cmp;
        }

        public void insert(@NotNull T x) {
            root = insertRec(root, x);
        }

        // 1.a
        @NotNull
        protected Node insertRec(@Nullable Node n, @NotNull T x) {
            if (n == null)
                return new Node(x);
            int r = cmp.compare(x, n.data);
            if (r < 0)
                n.left = insertRec(n.left, x);
            else if (r > 0)
                n.right = insertRec(n.right, x);
            return n;
        }

        // 1.b.i
        protected void dfsInOrder(@Nullable Node n, @NotNull Collection<T> out) {
            if (n != null) {
                dfsInOrder(n.left, out);
                out.add(n.data);
                dfsInOrder(n.right, out);
            }
        }

        // 1.b.ii
        @NotNull
        @Override
        public Iterator<T> iterator() {
            Collection<T> c = new ArrayList<>();
            dfsInOrder(root, c);
            return c.iterator();
        }

        // 1.c
```

```java
        @Nullable
        public T min() {
            if (root == null) return null;
            Node n = root;
            while(n.left != null) n = n.left;
            return n.data;
        }

        @Nullable
        public T max() {
            if (root == null) return null;
            Node n = root;
            while(n.right != null) n = n.right;
            return n.data;
        }


        protected class Node {
            @NotNull
            private final T data;
            @Nullable
            protected Node left, right;

            protected Node(@NotNull T data, @Nullable Node left, @Nullable Node right) {
                this.data = data;
                this.left = left;
                this.right = right;
            }

            protected Node(@NotNull T data) {
                this(data, null, null);
            }
        }

        @Override
        @NotNull
        public String toString() {
            StringBuilder sb = new StringBuilder();
            for (T x : this) {
                sb.append(x);
                sb.append(" ");
            }
            return sb.toString();
        }
    }

    // 1.d
    public static class BST2<T extends Comparable<? super T>> extends BST<T> {
        public BST2() {
            super(Comparable::compareTo);
        }
    }

    public static void main(String[] args) {
        BST<Integer> t = new BST2<>();
        Random rnd = new Random();
        for (int i = 0; i < 20; ++i)
```

```
            t.insert(rnd.nextInt(100));
        System.out.println(t);
        System.out.printf("min = %d\nmax = %d\n", t.min(), t.max());
    }

}
```

4.2.8 Alberi binari (2.1.9)

```
// Questo sorgente contiene le soluzioni dell'esame scritto di PO2 del 13/9/2022 per ciò che riguarda il quesito 1,
// ovvero l'esercizio in Java.
// Il quesito 2 riguardante C++ è in una Solution per Visual Studio a parte.

import org.jetbrains.annotations.NotNull;
import org.jetbrains.annotations.Nullable;

import java.util.ArrayList;
import java.util.Collection;
import java.util.Iterator;
import java.util.concurrent.BlockingQueue;

public class Es1 {

    // 1.d

    public static void main(String[] args) {
        TreeNode<Integer> t1 =
            TreeNode.lr(1,
                TreeNode.lr(2,
                    TreeNode.v(3),
                    TreeNode.v(4)),
                TreeNode.r(5,
                    TreeNode.lr(6,
                        TreeNode.v(7),
                        TreeNode.v(8))));

        TreeNode<Integer> t2 =
            TreeNode.lr(1,
                TreeNode.r(5,
                    TreeNode.lr(6,
                        TreeNode.v(7),
                        TreeNode.v(8))),
                TreeNode.lr(2,
                    TreeNode.v(3),
                    TreeNode.v(4)));


        // test pretty printer
        System.out.println("pretty printer:");
        System.out.println("t1: " + t1);
        System.out.println("t2: " + t2);

        // test equality
        System.out.print("equality: ");
        System.out.println(t1.equals(t2) + ", " + (t1.left != null ? t1.left.equals(t2.right) : ""));

```

```java
        // test iterator
        System.out.print("iterator: ");
        for (Integer n : t1) {
            System.out.printf("%d ", n);
        }
        System.out.println();

    }

    public static class TreeNode<T> implements Iterable<T> {

        @NotNull
        private final T data;
        @Nullable
        private final TreeNode<T> left, right;
        @Nullable
        private TreeNode<T> parent = null;

        // 1.c

        public TreeNode(@NotNull T data, @Nullable TreeNode<T> left, @Nullable TreeNode<T>
        ↪  right) {
            this.data = data;
            this.left = left;
            this.right = right;
            if (left != null) left.parent = this;
            if (right != null) right.parent = this;
        }

        // i seguenti pseudo-costruttori aiutano a costruire alberi in modo più succinto e
        ↪  controllato rispetto
        // all'innestamento dei costruttori

        // solo ramo sinistro
        public static <T> TreeNode<T> l(@NotNull T data, @NotNull TreeNode<T> left) {
            return new TreeNode<>(data, left, null);
        }

        // solo ramo destro
        public static <T> TreeNode<T> r(@NotNull T data, @NotNull TreeNode<T> right) {
            return new TreeNode<>(data, null, right);
        }

        // ramo sinistro e destro
        public static <T> TreeNode<T> lr(@NotNull T data, @NotNull TreeNode<T> left, @NotNull
        ↪  TreeNode<T> right) {
            return new TreeNode<>(data, left, right);
        }

        // foglia
        public static <T> TreeNode<T> v(@NotNull T data) {
            return new TreeNode<>(data, null, null);
        }

        // 1.b

        @Override
```

```java
    public boolean equals(@Nullable Object o) {
        if (o instanceof TreeNode) {
            BlockingQueue<Integer> b;

            TreeNode<T> t = (TreeNode<T>) o;
            return areEqual(data, t.data) && areEqual(left, t.left) && areEqual(right, t.right);
        }
        return false;
    }

    private static boolean areEqual(@Nullable Object a, @Nullable Object b) {
        return a == b || (a != null && a.equals(b));
        //return Objects.equals(a, b);        // alternativamente si può usare questo metodo del JDK
    }

    // 1.e

    @Override
    @NotNull
    public String toString() {
        return String.format("%s%s%s", data, left != null ? String.format("(%s)", left) : "", right != null ?
            String.format("[%s]", right) : "");
    }

    // 1.a

    @Override
    public Iterator<T> iterator() {
        return iterator_easy();  // stub ad una delle due implementazione; cambiare lo stub per testare l'altra
    }


    // questo è l'implementazione facile, suggerita pubblicamente dal docente in classe duranto l'appello del 13/9/22
    private Iterator<T> iterator_easy() {
        Collection<T> r = new ArrayList<>();
        dfs(r);
        return r.iterator();
    }

    private void dfs(Collection<T> c) {
        c.add(data);
        if (left != null) left.dfs(c);
        if (right != null) right.dfs(c);
    }

    // questo è l'implementazione ottimizzata, che attraversa l'albero senza liste d'appoggio
    private Iterator<T> iterator_opt() {
        return new Iterator<>() {
            @Nullable
            private TreeNode<T> current = TreeNode.this;

            @Override
```

```java
                public boolean hasNext() {
                    return current != null;
                }

                @Nullable
                private static <T> TreeNode<T> getNextNode(@NotNull TreeNode<T> n) {
                    if (n.left != null)
                        return n.left;
                    else if (n.right != null)
                        return n.right;
                    else {
                        while (n.parent != null) {
                            final TreeNode<T> last = n;
                            n = n.parent;
                            if (n.right != null && n.right != last)
                                return n.right;
                        }
                        return null;
                    }
                }

                @Override
                @NotNull
                public T next() {
                    T r = current.data;
                    current = getNextNode(current);
                    return r;
                }
            };
        }

    }

}
```

4.2.9 SkippableArrayList (2.1.10)

```java

// Questo sorgente contiene le soluzioni dell'esame scritto di PO2 del 1/7/2022 per ciò che riguarda i quesiti 1-2, ovvero le domande che coinvolgono Java.
// Il quesito 3 riguardante C++ è in un progetto Visual Studio a parte, non qui.
// Il codice qui esposto è Java 8+.

import java.util.ArrayList;
import java.util.Iterator;
import java.util.function.Function;

public class Es1 {

    // 1.a
    public interface Predicate<T> extends Function<T, Boolean> {}

    // 1.b
    public interface Either<T> {
        T onSuccess(T x);
        void onFailure(T x) throws Exception;
    }

```

```java
    // 1.c
    public static class SkippableArrayList<E> extends ArrayList<E> {
        public Iterator<E> iterator(Predicate<E> p, Either<E> f) {
            final Iterator<E> it = super.iterator();    // può anche essere un campo privato della anonymous class
            return new Iterator<E>() {
                @Override
                public boolean hasNext() {
                    return it.hasNext();
                }

                @Override
                public E next() {
                    E x = it.next();
                    if (p.apply(x))
                        return f.onSuccess(x);
                    else {
                        try {
                            f.onFailure(x);
                        }
                        catch (Exception e) {
                            e.printStackTrace();    // si può anche non fare niente dentro il catch, è indifferente
                        }
                        return x;
                    }
                }
            };
        }
    }

}
```

4.2.10 Figure Geometriche (2.1.11)

```java
import java.util.*;
import java.util.function.BiFunction;
import java.util.function.Function;

// Questo sorgente contiene le soluzioni dell'esame scritto di PO2 del 3/6/2022 per ciò che riguarda i quesiti 1-5, ovvero le domande che coinvolgono Java.
// Il quesito 6 riguardante C++ è in una Solution per Visual Studio a parte, non qui.

public class Appello_3_6_22 {

    public static <T, State> State fold(Iterable<T> i, final State st0, BiFunction<State, T, State> f) {
        State st = st0;
        for (final T e : i) {
            st = f.apply(st, e);
        }
        return st;
    }

    // 1.a
    public static <T> double sumBy(Iterable<T> i, Function<T, Double> f) {
        return fold(i, 0., (r, x) -> r + f.apply(x));
    }
```

```java

    // 1.b
    public static <T> int compareBy(T s1, T s2, Function<T, Double> f) {
        return Double.compare(f.apply(s1), f.apply(s2));
    }

    public static class Edge implements Comparable<Edge> {
        private final double len;

        public Edge(double len) {
            this.len = len;
        }

        public double length() {
            return len;
        }

        // 2.a
        @Override
        public int compareTo(Edge s) {
            return compareBy(this, s, Edge::length);
        }
    }

    public interface Surface extends Comparable<Surface> {
        double area();

        double perimiter();

        // 2.b
        @Override
        default int compareTo(Surface s) {
            return compareBy(this, s, Surface::area);
        }
    }

    public interface Polygon extends Surface, Iterable<Edge> {
        // 2.c
        @Override
        default double perimiter() {
            return sumBy(this, Edge::length);
        }
    }

    public interface Solid extends Comparable<Solid> {
        double outerArea();

        double volume();

        // 2.d
        default int compareTo(Solid s) {
            return compareBy(this, s, Solid::volume);
        }
    }

    public interface Polyhedron<P extends Polygon> extends Solid, Iterable<P> {
        // 2.e
```

```java
        @Override
        default double outerArea() {
            return sumBy(this, P::area);
        }
    }

    // 3.a
    public static class Sphere implements Solid {
        private final double radius;

        public Sphere(double radius) {
            this.radius = radius;
        }

        @Override
        public double outerArea() {
            return 4. * Math.PI * radius * radius;
        }

        @Override
        public double volume() {
            return 4. / 3. * Math.PI * radius * radius * radius;
        }
    }

    // 3.b
    public static class Cilinder implements Solid {
        private final double radius, height;

        public Cilinder(double radius, double height) {
            this.radius = radius;
            this.height = height;
        }

        @Override
        public double outerArea() {
            double b = Math.PI * radius * radius, l = 2. * Math.PI * radius * height;   // hanno capito solo laterale
            return 2. * b + l;
        }

        @Override
        public double volume() {
            return Math.PI * radius * radius * height;
        }
    }

    // 3.c
    public static class Rectangle implements Polygon {
        private final double width, height;

        public Rectangle(double width, double height) {
            this.width = width;
            this.height = height;
        }

        @Override
```

```java
        public double area() {
            return width * height;
        }

        @Override
        public Iterator<Edge> iterator() {
            Edge w = new Edge(width), h = new Edge(height);
            return List.of(w, h, w, h).iterator();
        }
    }

    // 3.d
    public static class Square extends Rectangle {
        public Square(double side) {
            super(side, side);
        }
    }

    public static class Parallelepiped implements Polyhedron<Rectangle> {
        protected double width, height, depth;

        public Parallelepiped(double width, double height, double depth) {
            this.width = width;
            this.height = height;
            this.depth = depth;
        }

        // 4.a
        @Override
        public double volume() {
            return width * height * depth;
        }

        @Override
        public Iterator<Rectangle> iterator() {
            Rectangle r1 = new Rectangle(width, height), r2 = new Rectangle(width, depth), r3 = new Rectangle(height, depth);
            return List.of(r1, r2, r3, r1, r2, r3).iterator();
        }
    }

    // 4.b
    public static class Cube extends Parallelepiped {
        public Cube(double side) {
            super(side, side, side);
        }
    }


    public static void main(String[] args) {
        // 4.d
        {
            int facet_cnt = 1;
            // questo foreach non compila perché Cube è sottotipo di Iterable<Rectangle>, non di Iterable<Square>
            // si badi che NON è possibile co-variare il tipo di ritorno del metodo iterator() di Cube in modo che si specializzi in Iterator<Square>
```

```java
            // perché è sound co-variare il tipo più esterno di un tipo parametrico, ma non il type argument
            // for (Square sq : new Cube(10.)) {
            for (Rectangle sq : new Cube(10.)) {  // così invece compilerebbe
                int side_cnt = 1;
                for (Edge e : sq) {
                    System.out.printf("side #%d/%d = %f\n", side_cnt++, facet_cnt, e.length());
                }
                ++facet_cnt;
            }
        }

        // 5
        {
            Cube c1 = new Cube(1.), c2 = new Cube(2.);
            Parallelepiped p1 = new Parallelepiped(1., 2., 3.), p2 = new Parallelepiped(2., 3., 4.);
            List<Polyhedron<? extends Rectangle>> polys = new ArrayList<>(List.of(c1, c2, p1, p2));

            // per testare rapidamente i risultati di queste sort, si usi debugger
            Collections.sort(polys); // c1
            Collections.sort(polys, (x, y) -> compareBy(x, y, Polyhedron::outerArea)); // c1
            Collections.sort(polys, (x, y) -> compareBy(x, y, (p) -> p.outerArea())); // c1
//            Collections.sort(polys, (x, y) -> compareBy(x, y, (r) -> r.perimiter())); // non compila
            Collections.sort(polys, (x, y) -> compareBy(x, y, new Function<>() { // c1
                @Override
                public Double apply(Polyhedron<? extends Rectangle> r) {
                    return r.volume();
                }
            }));
//            Collections.sort(polys, (x, y) -> Double.compare(sumBy(x, Square::perimiter), // non compila
//                                                              sumBy(y, Rectangle::perimiter)));
        }
    }
}
```

4.2.11 Artisti (2.1.12)

```java
import java.util.*;

public class Es1 {

    public interface Equatable<T> {
        boolean equalsTo(T x);
    }

    public static class Person<P extends Person<P>> implements Equatable<P> {

        public final String name;
        public final int age;

```

```java
        public Person(String name, int age) {
            this.name = name;
            this.age = age;
        }

        // 1.a
        @Override
        public boolean equals(Object o) {
            // self check
            if (this == o)
                return true;
            // null check
            if (o == null)
                return false;
            // type check
            if (getClass() != o.getClass())
                return false;
            // field comparison
            return equalsTo((P) o);    // deleghiamo al metodo equalsTo(P) il confronto tipato tra i campi
        }

        // 1.b
        @Override
        public boolean equalsTo(P other) {
            return name.equals(other.name) && age == other.age;
        }

        @Override
        public String toString() {
            return name;
        }

    }

    public static class Artist extends Person<Artist> {

        public final Hair hair;

        public Artist(String name, int age, Hair hair) {
            super(name, age);
            this.hair = hair;
        }

        // 1.d RISPOSTA: 4
        // 1.e
        @Override
        public boolean equalsTo(Artist other) {
            return super.equalsTo(other) && hair.equals(other.hair);
        }

    }

    public enum Color {
        BROWN, DARK, BLONDE, RED, GRAY;
    }

```

```java
    public static class Hair implements Equatable<Hair> {
        public final int length;
        public final Set<Color> colors;

        public Hair(int length, Set<Color> colors) {
            this.colors = colors;
            this.length = length;
        }

        // 1.c
        @Override
        public boolean equals(Object o) {   // uguale a Person.equals(Object)
            if (this == o)
                return true;
            if (o == null)
                return false;
            if (getClass() != o.getClass())
                return false;
            return equalsTo((Hair) o);
        }

        // 1.c
        @Override
        public boolean equalsTo(Hair x) {
            return colors.equals(x.colors) && length == x.length;
        }
    }


    public static void print(String fmt, Object... o) {
        System.out.println(String.format(fmt, o));
    }

    public static void main(String[] args) {
        Person alice = new Person("Alice", 23),
                david = new Artist("Bowie", 69, new Hair(75, Set.of(Color.RED, Color.BROWN, Color.GRAY)));

        Artist morgan = new Artist("Morgan", 47, new Hair(20, Set.of(Color.GRAY, Color.DARK))),
                madonna = new Artist("Madonna", 60, new Hair(50, Set.of(Color.BLONDE)));

        // 1.g (lanciate il programma per printare le risposte booleane)
        List<Boolean> bs = Arrays.asList(
                alice.equals(null),         // false
                alice.equals(alice),        // true
                //null.equals(david),       // alcuni compilatori rifiutano null.metodo(), altri lo accettano ma a runtime viene lanciato NullPointerException
                alice.equals(david),        // false
                alice.equalsTo(morgan),     // false
                morgan.equals(morgan),      // true
                morgan.equals(madonna),     // false
                morgan.equals(david),       // false
                //morgan.equalsTo(david),   // non compila
                david.equalsTo(morgan),     // false
                madonna.equals(3)           // false
                //, madonna.equalsTo("Madonna") // non compila
        );
```

```java
        print("[1.g] %s", bs);

        // 1.h
        List<Artist> artists = Arrays.asList((Artist) david, morgan, madonna);
        List<Person> persons = Arrays.asList(alice, david, morgan, madonna);

        // 1.h.i
        Artist a = Collections.max(artists, new Comparator<Artist>() {
            @Override
            public int compare(Artist a, Artist b) {
                return a.hair.length * a.hair.colors.size() - b.hair.length *
                ↪  b.hair.colors.size();
            }
        });
        print("[1.h.i] %s", a);

        // 1.h.ii
        Person b = Collections.max(persons, new Comparator<Person>() {
            @Override
            public int compare(Person a, Person b) {
                return a.name.compareTo(b.name);
            }
        });
        print("[1.h.ii] %s", b);

        // 1.h.iii: RISPOSTA: 2
        Artist c = Collections.max(artists, new Comparator<Person>() {
            public int compare(Person a, Person b) {
                return a.age - b.age;
            }
        });

    }


}
```

4.2.12 Iteratori (2.1.13)

```java
import java.util.*;

public class Es2 {


    // 2.a
    @FunctionalInterface
    public interface BiFunction<A, B, C> {
        C apply(A a, B b);
    }

    // 2.b
    public static class Pair<A, B> {
        public final A fst;
        public final B snd;

        public Pair(A a, B b) {
            this.fst = a;
            this.snd = b;
```

```java
        }
    }

    // 2.c
    public static class Triple<A, B, C> extends Pair<A, B> {
        public final C trd;

        public Triple(A a, B b, C c) {
            super(a, b);
            this.trd = c;
        }
    }

    // 2.d
    public static <A, B, C> Iterator<Triple<A, B, C>> evalIterator(Iterator<Pair<A, B>> it,
    ↪ BiFunction<A, B, C> f) {
        return new Iterator<>() {
            @Override
            public boolean hasNext() {
                return it.hasNext();
            }

            @Override
            public Triple<A, B, C> next() {
                Pair<A, B> p = it.next();
                return new Triple<>(p.fst, p.snd, f.apply(p.fst, p.snd));
            }
        };
    }

}
```

4.2.13 Sequenza di Fibonacci (2.1.14)

```java
import java.util.HashMap;
import java.util.Iterator;
import java.util.Map;

public class Es3 {

    // 3.a
    public static class FiboSequence implements Iterable<Integer> {

        private final int max;
        private final Map<Integer, Integer> cache = new HashMap<>();

        public FiboSequence(int max) {
            this.max = max;
        }

        @Override
        public Iterator<Integer> iterator() {
            return new Iterator<>() {
                private int i = 0;

                @Override
                public boolean hasNext() {
                    return i < max;
```

```
                }

                @Override
                public Integer next() {
                    return fib(i++);
                }

                private int fib(int n) {
                    if (n < 2) return 1;
                    else {
                        Integer x = cache.get(n);
                        if (x != null) return x;
                        else {
                            int r = fib(n - 1) + fib(n - 2);
                            cache.put(n, r);    // si commenti questo statement per
                                                // disabilitare la cache e vedere la differenza
                                                // enorme di tempi di calcolo
                            return r;
                        }
                    }
                }
            };
        }
    }

    // si lanci questo main e si osservi quanto velocemente la macchina produce 100 numeri di fibonacci:
    // se non ci fosse la cache sarebbe drammaticamente più lento a causa delle continue ricorsioni
    public static void main(String[] args) {
        for (int n : new FiboSequence(100)) {
            System.out.println(n);
        }
    }


    // 3.b
    public static void main__desugared() {
        // questo for senza il terzo statement è equivalente ad un while
        for (Iterator<Integer> it = new FiboSequence(100).iterator(); it.hasNext(); ) {
            int n = it.next();
            System.out.println(n);
        }
    }


    // 3.c
    public static class GlobalFiboSequence implements Iterable<Integer> {

        private final int max;
        private final static Map<Integer, Integer> cache = new HashMap<>(); // è tutto uguale a
                                                                            // FiboSequence eccetto per
                                                                            //la cache che è statica

        public GlobalFiboSequence(int max) {
```

```java
            this.max = max;
        }

        @Override
        public Iterator<Integer> iterator() {
            return new Iterator<>() {
                private int i = 0;

                @Override
                public boolean hasNext() {
                    return i < max;
                }

                @Override
                public Integer next() {
                    return fib(i++);
                }

                private int fib(int n) {
                    if (n < 2) return 1;
                    else {
                        Integer x = cache.get(n);
                        if (x != null) return x;
                        else {
                            int r = fib(n - 1) + fib(n - 2);
                            cache.put(n, r);    // si commenti questo statement per disabilitare la cache
                                                // e vedere la differenza enorme di tempi di calcolo
                            return r;
                        }
                    }
                }
            };
        }
    }

}
```

4.2.14 Random (2.1.15)

```java
import org.jetbrains.annotations.NotNull;
import org.jetbrains.annotations.Nullable;

import java.util.Iterator;
import java.util.Random;

public class Es7 {

    // 7.b
    public static class RandomSingleton {
        private RandomSingleton() {
        }

        @Nullable
        private static Random instance;

        @NotNull
```

```java
        private static Random instance() {
            if (instance == null)
                instance = new Random();
            return instance;
        }

        // fare un setter per cambiare il seed del PRNG è uno dei modi possibili per riprodurre
        // il comportamento originale della classe Random del JDK. Normalmente, per avere un
        // seed specifico è necessario costruire un oggetto Random tramite l'apposito
        // costruttore. Nella nostra conversione a singleton rappresentiamo invece questo
        // aspetto come un setter che internamente reistanzia il singleton in maniera
        // trasparente all'utente.
        public void setSeed(int seed) {
            instance = new Random(seed);
        }

        public static int nextInt() {
            return instance().nextInt();
        }

        public static boolean nextBoolean() {
            return instance().nextBoolean();
        }

        public static double nextDouble() {
            return instance().nextDouble();
        }

    }

    // 7.c
    public static class RandomIterator implements Iterator<Integer> {
        private int len;

        public RandomIterator(int len) {
            this.len = len;
        }

        @Override
        public boolean hasNext() {
            return len > 0;
        }

        @Override
        public Integer next() {
            --len;
            return RandomSingleton.nextInt();
        }
    }

}
```

4.2.15 FancyArrayList (2.1.16)

```java
import java.util.ArrayList;
import java.util.Iterator;

public class Es10 {
```

```java

    public static class FancyArrayList<E> extends ArrayList<E> {

        // 10.a
        @FunctionalInterface
        public interface Function<A, B> {
            B apply(A a);
        }

        // 10.b
        public Iterator<E> iterator(int step, Function<E, E> f) {
            return new Iterator<>() {
                private int pos = step > 0 ? 0 : size() - 1;

                @Override
                public boolean hasNext() {
                    return pos >= 0 && pos < size();
                }

                @Override
                public E next() {
                    E r = get(pos);
                    pos += step;
                    return f.apply(r);
                }
            };
        }

        // 10.c.i
        @Override
        public Iterator<E> iterator() {
            return iterator(1, (x) -> x);   // si può fare l'identità con la lambda (solo Java 8+)
        }

        // 10.c.ii
        public Iterator<E> backwardIterator() {
            return iterator(-1, new Function<>() {  // oppure con una anonymous class semplicissima
                @Override
                public E apply(E x) {
                    return x;
                }
            });
        }

        // 10.d
        // bisogna chiamare questo metodo con un nome diverso da iterator()
        // altrimenti il sorgente non compila
        public Iterator<E> iterator__refactored(int step, Function<E, E> f) {
            return new FancyIterator<>(this, step, f);
        }

        // questa classe è il refactoring della anonynmous class dell'esercizio 10.b:
        // la differenza è che conserva la enclosing instance esplicitamente
        private static class FancyIterator<E> implements Iterator<E> {
            private FancyArrayList<E> enclosing;
```

```java
            private Function<E, E> f;
            private int step, pos;

            public FancyIterator(FancyArrayList<E> parent, int step, Function<E, E> f) {
                this.enclosing = parent;
                this.step = step;
                this.f = f;
                pos = step > 0 ? 0 : parent.size() - 1;
            }

            @Override
            public boolean hasNext() {  // questo metodo è uguale
                return pos >= 0 && pos < enclosing.size();
            }

            @Override
            public E next() {
                E r = enclosing.get(pos);
                pos += step;
                return f.apply(r);
            }
        }

    }

}
```

4.2.16 Minimo e Massimo di una Lista (2.1.17)

```java
import java.util.Comparator;
import java.util.List;

public class Es11 {

    // 11.a
    // versione con campi pubblici immutabili, ma si potevano fare anche i getter volendo
    public static class Pair<A, B> {
        public final A a;
        public final B b;
        public Pair(A a, B b) {
            this.a = a;
            this.b = b;
        }
    }

    // 11.b
    public static <E> Pair<E, E> findMinAndMax(List<? extends E> l, Comparator<E> c) {
        E min = l.get(0), max = min;
        for (E x : l) {
            if (c.compare(x, max) > 0) max = x;
            if (c.compare(x, min) < 0) min = x;
        }
        return new Pair<>(min, max);
    }

    // 11.c
    public static <E extends Comparable<E>> Pair<E, E> findMinAndMax(List<? extends E> l) {
        return findMinAndMax(l, new Comparator<>() {
```

```
            @Override
            public int compare(E a, E b) {
                return a.compareTo(b);
            }
        });
    }

}
```

4.2.17 Hanna Barbera (2.1.18)

```
public class Es12 {

    // tipi dati dal testo

    public interface Food {
        int getWeight();
    }

    public interface Animal extends Food {
        void eat(Food f);
    }

    public static class Mouse implements Animal {
        private int weight;

        public Mouse(int weight) {
            this.weight = weight;
        }

        @Override
        public void eat(Food f) {
            weight += f.getWeight() / 3;
        }

        @Override
        public int getWeight() {
            return weight;
        }
    }

    public static class Cat implements Animal {
        private int weight;

        public Cat(int weight) {
            this.weight = weight;
        }

        @Override
        public void eat(Food f) {
            weight += f.getWeight() / 10;
        }

        @Override
        public int getWeight() {
            return weight;
        }
    }
```

```java

    public static class Cheese implements Food {
        @Override
        public int getWeight() {
            return 300;
        }
    }


    // 12.a
    // le interfacce Food e Animal non serve rifattorizzarle, sono a posto così
    public abstract class AbstractAnimal implements Animal {
        private int weight, div;

        protected AbstractAnimal(int weight, int div) {
            this.weight = weight;
        }

        @Override
        public void eat(Food f) {
            weight += f.getWeight() / div;
        }

        @Override
        public int getWeight() {
            return weight;
        }
    }

    public class Cat extends AbstractAnimal {
        protected Cat(int weight, int div) {
            super(weight, 5);   // valori del tema A
        }

        public class Mouse extends AbstractAnimal {
            protected Mouse(int weight, int div) {
                super(weight, 2);
            }
        }


        // 12.b.i
        public static void main(String[] args) {
            Animal tom = new Cat(200);     // costanti del tema A
            Animal jerry = new Mouse(10);
            jerry.eat(new Cheese());
            jerry.eat(new Food() {
                @Override
                public int getWeight() {
                    return 10;
                }
            });
            tom.eat(jerry);
            System.out.println(String.format("Tom now weights %d", tom.getWeight())); //b.ii 208

        }
    }
```

4.3 Soluzione esercizi di progettazione e implementazione - C++

4.3.1 Pair (2.2.1)

```
export module pairs;

import <string>;
import <functional>;


export
template <class A, typename B>
class pair
{
        template <class C, typename D> friend class pair;        // necessario per il copy
        ↪  constructor templatizzato

private:
        A first;
        B second;

public:
        // 2.a
        pair() : first(), second() {}
        pair(const A& a, const B& b) : first(a), second(b) {}
        pair(const pair<A, B>& p) : first(p.first), second(p.second) {}

        pair<A, B>& operator=(const pair<A, B>& p)
        {
                first = p.first;
                second = p.second;
                return *this;
        }

        // 2.c
        template <class C, typename D>
        pair(const pair<C, D>& p) : first(p.first), second(p.second) {}


        // 2.b
        pair<A, B> operator++(int)
        {
                pair<A, B> tmp(*this);
                first++;
                second++;
                return tmp;
        }

        pair<A, B>& operator++()
        {
                ++first;
                ++second;
                return *this;
        }

        bool operator==(const pair<A, B>& p) const
        {
                return first == p.first && second == p.second;
```

```
54          }
55
56          bool operator!=(const pair<A, B>& p) const
57          {
58                  return !(*this == p);
59          }
60
61          // altri operatori aritmentici ed in-place sono analoghi
62          pair<A, B> operator+(const pair<A, B>& p) const
63          {
64                  return pair<A, B>(first + p.first, second + p.second);
65          }
66
67          pair<A, B>& operator+=(const pair<A, B>& p)
68          {
69                  first += p.first;
70                  second += p.second;
71                  return *this;
72          }
73
74          const A& fst() const
75          {
76                  return first;
77          }
78
79          A& fst()
80          {
81                  return first;
82          }
83
84          const B& snd() const
85          {
86                  return second;
87          }
88
89          B& snd()
90          {
91                  return second;
92          }
93
94  };
95
96
97  export int main()
98  {
99          pair<int, int> p1(4, 5);
100         pair<int, int> p2(p1);
101
102         pair<std::string, bool> p3("ciao", true);
103         pair<double, double> p4(p1);
104
105         p1 = p2;
106
107         int n = p1.fst();
108         p1.snd() = p1.snd() * 3;
109         p4 += p1;        // converte implicitamente il RV in un pair<double, double> tramite un
            ↪  conversion copy-constructor templatizzato
```

```
        return 0;
}
```

4.3.2 Matrici (2.2.2)

```
// Questo sorgente contiene le soluzioni dell'esame scritto di PO2 del 3/6/2022 per ciò che riguarda il quesito 6, ovvero la domanda che coinvolge C++.
// I quesiti 1-5 riguardanti Java sono in un progetto IntelliJ a parte, non qui.
// Il codice C++ qui esposto è standard C++ vanilla (a.k.a. C++03), sebbene il progetto VS sia configurato con il compilatore di default C++14

#include <iostream>
#include <vector>

using namespace std;

// 6
template <class T>
class matrix
{
private:
        size_t cols;
        vector<T> v;

public:
        matrix() : cols(0), v() {}
        matrix(size_t rows, size_t cols_) : cols(cols_), v(rows * cols) {}
        matrix(size_t rows, size_t cols_, const T& v) : cols(cols_), v(rows* cols, v) {}
        matrix(const matrix<T>& m) : cols(m.cols), v(m.v) {}

        typedef T value_type;
        typedef typename vector<T>::iterator iterator;
        typedef typename vector<T>::const_iterator const_iterator;

        matrix<T>& operator=(const matrix<T>& m)
        {
                v = m.v;
                return *this;
        }

        T& operator()(size_t i, size_t j)
        {
                return v[i * cols + j];
        }

        const T& operator()(size_t i, size_t j) const
        {
                return (*this)(i, j);
        }

        iterator begin()
        {
                return v.begin();
        }

        iterator end()
        {
```

```
                return v.end();
        }

        const_iterator begin() const
        {
                return begin();
        }

        const_iterator end() const
        {
                return end();
        }
};


int main()
{
        matrix<double> m1;          // non inizializzata
        matrix<double> m2(10, 20);  // 10*20 inizializzata col default constructor di double
        matrix<double> m3(m2);      // costruita per copia
        m1 = m2;                    // assegnamento
        m3(3, 1) = 11.23;           // operatore di accesso come left-value

        for (typename matrix<double>::iterator it = m1.begin(); it != m1.end(); ++it) {
                typename matrix<double>::value_type& x = *it;        // de-reference non-const
                x = m2(0,
                ↪  2);                                                                    //
                ↪  operatore di accesso come right-value
        }

        matrix<string> ms(5, 4, "ciao"); // 5*4 inizializzata col la stringa passata come terzo
        ↪  argomento
        for (typename matrix<string>::const_iterator it = ms.begin(); it != ms.end(); ++it)
                cout << *it;                                // de-reference const
}
```

4.3.3 Sommatoria (2.2.3)

```
// Questo sorgente contiene le soluzioni dell'esame scritto di PO2 del 5/9/2023 per ciò che
↪  riguarda il quesito 2, ovvero la domanda che coinvolge C++.
// I quesiti 1-5 riguardanti Java sono in un sorgente Java a parte, non qui.
// Il codice C++ qui esposto è standard C++ vanilla (a.k.a. C++03), sebbene il progetto VS sia
↪  configurato con il compilatore di default C++14

#include <iostream>
#include <vector>

using namespace std;

template <class Container>
ostream& operator<<(ostream& os, const Container& c)
{
        os << "[";
        for (typename Container::const_iterator it = c.begin(); it != c.end(); ++it)
                os << " " << *it;
        os << "]";
        return os;
}
```

```

template <class Container>
typename Container::value_type sum(const Container& c)
{
        typename Container::value_type r;
        for (typename Container::const_iterator it = c.begin(); it != c.end(); ++it)
        {
                r += *it;
        }
        return r;
}


int main()
{
}
```

4.3.4 Map (2.2.4)

```
// Scritto P02 10 9 24.cpp : This file contains the 'main' function. Program execution begins and
↪  ends there.
//

#include <iostream>
#include <functional>
#include <vector>
#include <type_traits>

using namespace std;

// 2.a
template <class InputIterator, class OutputIterator>
void map(InputIterator from, InputIterator to, OutputIterator out, function<typename
↪  OutputIterator::value_type(typename InputIterator::value_type)> f)
{
    while (from != to)
        *out++ = f(*from++);
}

// 2.b
template <class A, class B>
vector<B> map(const vector<A>& v, function<B(A)> f)
{
    vector<B> r(v.size());
    map(v.begin(), v.end(), r.begin(), f);
    return r;
}

// 2.c
namespace cpp03 {

    // 2.c.i
    template <class A, class B, class F>
    vector<B> map(const vector<A>& v, F f)
    {
        vector<B> r(v.size());
        for (int i = 0; i < v.size(); ++i)
            r[i] = f(v[i]);
```

```
        return r;
    }

    // 2.c.ii
    // no, non possono coesistere, perché sarebbero overload ambigui. Infatti ho dovuto metterli in un sotto-namespace a parte per far compilare questo sorgente.

    // 2.c.iii
    // bisogna usare i function object, cioè oggetti per cui è definito l'operatore di applicazione operator()
    // esempio:
    class myfunction {
    public:
        bool operator()(int n) { return n > 2; }
    };

    void test()
    {
        vector<int> v1{ 1, 2, 3, 4, 5 };
        vector<bool> v2 = map<int, bool>(v1, myfunction()); // senza le annotazioni esplicite dei template argument non compila (questo non è richiesto nell'esame perché è una cosa molto sottile)
    }
}

// main for testing
//

template <class T>
ostream& operator<<(ostream& os, const vector<T>& v)
{
    os << "[ ";
    for (auto it = v.begin(); it != v.end(); ++it)
        os << *it << ", ";
    os << "\b\b ]";
    return os;
}

int main()
{
    vector<int> v1{ 1, 2, 3, 4, 5 };
    vector<bool> v2(v1.size());
    map(v1.begin(), v1.end(), v2.begin(), [](int n) { return n > 2; });

    v2 = map<int, bool>(v1, [](int n) { return n > 2; }); // anche qui bisogna mettere i template argument espliciti

    cout << v1 << endl << v2 << endl;

    return 0;
}
```

4.3.5 Funzioni (2.2.5)

```
export module pairs;

import <iostream>;
import <functional>;
```

```
import <utility>;

using namespace std;

export
template <class X, class Y>
class fun_seq
{
private:
        const X a, b, dx;
        const function<Y(const X&)> f;

public:
        using value_type = pair<X, Y>;

        fun_seq(const X& _a, const X& _b, const X& _dx, const function<Y(const X&)>& _f) : a(_a), b(_b), dx(_dx), f(_f) {}

        class iterator
        {
        private:
                X x;
                fun_seq<X, Y> that;

        public:
                explicit iterator(const X& _x, const fun_seq<X, Y>& _that) : x(_x), that(_that) {}

                iterator(const iterator& it) : x(it.x), that(it.that) {}

                pair<X, Y> operator*() const
                {
                        return pair<X, Y>(x, that.f(x));
                }

                bool operator!=(const iterator& it)
                {
                        return !(x > it.x - that.dx && x < it.x + that.dx);
                }

                iterator operator++()
                {
                        iterator r(*this);
                        x += that.dx;
                        return r;
                }
        };

        iterator begin() const
        {
                return iterator(a, *this);
        }

        iterator end() const
        {
                return iterator(b, *this);
        }
```

```
};


export int main()
{
        fun_seq<double, double> sq1(-2., 2., 0.1, [](const double& x) { return x * x + 2 * x - 1; });
        for (pair<double, double> p : sq1)
                cout << "f(" << p.first << ") = " << p.second << endl;

        fun_seq<double, int> sq2(-2., 2., 0.1, [](const double& x) { return int(x * x + 2 * x - 1); });
        for (pair<double, int> p : sq2)
                cout << "f(" << p.first << ") = " << p.second << endl;

        return 0;
}
```

4.3.6 Curve (2.2.6)

```
#include <functional>
#include <iostream>
#include <utility>
#include <algorithm>

using namespace std;


using real = double;
using unary_fun = function<real(const real&)>;

#define RESOLUTION (10)


class curve
{

private:
        real a, b;
        unary_fun f;

public:
        curve(const real& a_, const real& b_, const unary_fun& f_) : f(f_), a(a_), b(b_) {}

        real get_dx() const { return (b - a) / RESOLUTION; }

        pair<real, real> interval() const { return pair<real, real>(a, b); }

        curve derivative() const
        {
                return curve(a, b, [&, dx = get_dx()](const real& x) {        // anche la capture [=] sarebbe stata sufficiente
                        const real dy = f(x + dx) - f(x);
                        return dy / dx;
                        });
        }
```

```
36
37          curve primitive() const
38          {
39                  return curve(a, b, [&, dx = get_dx()](const real& x) {        // oppure la [&]
40                          const real y = f(x);
41                          return y * dx;
42                          });
43          }
44
45          real integral() const
46          {
47                  const unary_fun& F = primitive();
48                  return F(b) - F(a);
49          }
50
51          real operator()(const real& x) const
52          {
53                  return f(x);
54          }
55
56          class iterator
57          {
58          private:
59                  const curve& c;
60                  real x;
61
62          public:
63                  iterator(const curve& c_, const real& x_) : c(c_), x(x_) {}
64
65                  pair<real, real> operator*() const
66                  {
67                          return pair<real, real>(x, c.f(x));
68                  }
69
70                  iterator& operator++()        // il pre-incremento modifica sè stesso e non
                    ↪  ritorna una copia
71                  {
72                          x += c.get_dx();
73                          return *this;
74                  }
75
76                  iterator operator++(int)        // il post-incremento fa una copia, modifica sé
                    ↪  stesso e ritorna la copia
77                  {
78                          auto r(*this);
79                          ++(*this);
80                          return r;
81                  }
82
83                  bool operator!=(const iterator& it) const
84                  {
85                          return fabs(x - it.x) >= c.get_dx();        // non si confrontano mai i
                            ↪  float direttamente con l'operatore di uguaglianza o disuguaglianza
86                  }
87          };
88
89          iterator begin() const
```

```
        {
                return iterator(*this, a);
        }

        iterator end() const
        {
                return iterator(*this, b + get_dx());
        }

};


ostream& operator<<(ostream& os, const curve& c)
{
        os << "dom = [" << c.interval().first << ", " << c.interval().second << "] dx = " << c.get_dx() << ": " << endl;
        /*for (const pair<real, real>& p : c)
        {
                const real& x = p.first, & y = p.second;
                os << "\tf(" << x << ") = " << y << endl;
        }*/
        for (curve::iterator it = c.begin(); it != c.end(); ++it)
        {
                const pair<real, real>& p = *it;
                const real& x = p.first, & y = p.second;
                os << "\tf(" << x << ") = " << y << endl;
        }
        return os << endl;
}

int main()
{
        curve f(-1., 1., [](const real& x) { return x * x; });

        cout << f << endl
                << f.derivative() << endl
                << f.primitive() << endl
                << f.primitive().derivative() << endl
                << f.derivative().primitive() << endl
                ;
        return 0;
}
```

4.3.7 Alberi binari (2.2.7)

```

// Questo sorgente contiene le soluzioni dell'esame scritto di PO2 del 1/7/2022 per ciò che riguarda il quesito 2, ovvero l'esercizio di C++.
// Il quesito 1 riguardante Java è in un progetto IntelliJ a parte, non qui.
// Il codice qui esposto è C++14.
// ATTENZIONE: il codice qui fornito è ricco di dettagli e complessità, allo scopo di fornire materiale di studio. La versione richiesta all'esame è molto più semplice.

#include <iostream>
#include <iterator>
#include <cstddef>
#include <vector>

```

```
using std::vector;

#define EASY_ITERATOR        // commentare questa macro per compilare la versione ottimizzata degli iteratori

template <class T>
class tree_node
{
public:
        T data;
        tree_node<T>* left, * right;

        static bool are_equal(const tree_node<T>* a, const tree_node<T>* b)
        {
                return a == b || (a != nullptr && b != nullptr && *a == *b);
        }

public:
        // 2.c

        tree_node() = default;
        tree_node(const tree_node<T>& t) = default;
        tree_node<T>& operator=(const tree_node<T>& t) = default;

        ~tree_node()
        {
                if (left != nullptr)
                {
                        delete left;
                        left = nullptr;
                }
                if (right != nullptr)
                {
                        delete right;
                        right = nullptr;
                }
        }

        tree_node(const T& v, tree_node<T>* l, tree_node<T>* r) : data(v), left(l), right(r)
        {
#                ifdef EASY_ITERATOR
                prepopulate();
#                else
                if (left != nullptr) left->parent = this;
                if (right != nullptr) right->parent = this;
#                endif
        }

        // 2.b

        bool operator==(const tree_node<T>& t) const
        {
                return data == t.data && are_equal(left, t.left) && are_equal(right, t.right);
        }

        // 2.a

```

```
        using value_type = T;


        // implementazione facile degli iteratori, suggerita pubblicamente dal docente durante l'appello del 13/9/22
        //

#        ifdef EASY_ITERATOR

        using const_iterator = typename vector<T>::const_iterator;
        using iterator = typename vector<T>::iterator;

private:
        // per motivi di semplicità ogni nodo ha un campo di tipo vector che viene popolato al volo per essere iterator
        // si faccia attenzione ad un particolare: per poter ritornare begin() ed end() dello stesso vector, bisogna conservarlo come membro del nodo
        vector<T> children;

        void dfs(vector<T>& v)
        {
                // perché non popoliamo direttamente il campo children? il motivo è molto sottile: ogni nodo ha un suo campo children, ma noi non vogliamo che ogni nodo popoli il proprio vector; noi vogliamo
                //   che quando decidiamo di popolare il campo di children di un certo nodo, allora viene popolato il vector di QUEL nodo
                // ogni nodo quindi ha un suo campo children che viene popolato con i SUOI sottorami: tutto ciò è uno spreco di spazio, certo, però permette di fare iteratori a partire da QUALUNQUE nodo
                //   in maniera semplice e immutabile
                v.push_back(data);
                if (left != nullptr) left->dfs(v);
                if (right != nullptr) right->dfs(v);
        }

        // questa viene chiamata dal costruttore: ogni nodo prepopola il proprio campo children
        void prepopulate()
        {
                dfs(children);
        }

public:

        const_iterator begin() const
        {
                return children.begin();
        }

        const_iterator end() const
        {
                return children.end();
        }

        iterator begin()
        {
                return children.begin();
        }
```

```
117
118         iterator end()
119         {
120                 return children.end();
121         }
122
123
124         // implementazione ottimizzata degli iteratori
125         //
126
127 #        else
128
129 private:
130         tree_node<T>* parent;
131
132         static tree_node<T>* get_next_node(const tree_node<T>* n) {
133                 if (n->left != nullptr)
134                         return n->left;
135                 else if (n->right != nullptr)
136                         return n->right;
137                 else {
138                         while (n->parent != nullptr) {
139                                 const tree_node<T>* last = n;
140                                 n = n->parent;
141                                 if (n->right != nullptr && n->right != last)
142                                         return n->right;
143                         }
144                         return nullptr;
145                 }
146         }
147
148 public:
149         // iteratore non-const
150         class my_iterator
151         {
152                 friend class my_const_iterator;        // questo serve perché altrimenti
                 ↪  my_const_iterator non può accedere al campo current di my_iterator
153
154         private:
155                 tree_node<T>* current;
156
157         public:
158                 using iterator_category = std::forward_iterator_tag;        // questo member type
                 ↪  indica che si tratta di un ForwardIterator, si veda la doc di STL per i
                 ↪  dettagli
159                 using difference_type = std::ptrdiff_t;
160                 using value_type = T;
161                 using pointer = T*;
162                 using reference = T&;
163
164                 my_iterator() = default;
165                 my_iterator(const my_iterator& i) = default;
166                 my_iterator& operator=(const my_iterator& i) = default;
167
168                 my_iterator(tree_node<T>* t) : current(t) {}
169
170                 reference operator*() { return current->data; }
```

```
171                 pointer operator->() { return &current->data; }
172
173                 my_iterator& operator++()
174                 {
175                         current = get_next_node(current);
176                         return *this;
177                 }
178
179                 my_iterator operator++(int)
180                 {
181                         my_iterator r(*this);
182                         current = get_next_node();
183                         return r;
184                 }
185
186                 bool operator==(const my_iterator& i) const { return current == i.current; }
187                 bool operator!=(const my_iterator& i) const { return !(*this == i); }
188
189         };
190
191         // interatore const
192         // si noti come sono praticamente uguali a parte il fatto che gesticono un nodo const
            ↪  oppure no, con conseguente impatto in tutti i tipi di ritorno dei vari operatori
193         // questa replicazione di codice sarebbe evitabile solamente tramite un complesso uso dei
            ↪  template, troppo complesso per questo corso
194         // chi è interessato a sapere come evitare questa duplicazione di codice dovuta a const
            ↪  vs. non-const può approfondire qui:
            ↪  https://stackoverflow.com/questions/765148/how-to-remove-constness-of-const-iterator
195         class my_const_iterator
196         {
197         private:
198                 const tree_node<T>* current;
199
200         public:
201                 using iterator_category = std::forward_iterator_tag;
202                 using difference_type = std::ptrdiff_t;
203                 using value_type = const T;
204                 using pointer = const T*;
205                 using reference = const T&;
206
207                 my_const_iterator() = default;
208                 my_const_iterator(const my_const_iterator& i) = default;
209                 my_const_iterator& operator=(const my_const_iterator& i) = default;
210
211                 // questo costruttore è molto interessante: permette di costruire un
                    ↪  my_const_iterator dato un my_iterator: in altre parole possiamo convertire un
                    ↪  iteratore non-const in uno const
212                 // il motivo per cui è necessario è se chiamiamo begin() su un tree_node
                    ↪  non-const ma vogliamo un const_iterator perché lo leggiamo soltanto
213                 my_const_iterator(const my_iterator& i) : current(i.current) {}
214
215                 my_const_iterator(const tree_node<T>* t) : current(t) {}
216
217                 reference operator*() const        { return current->data;        }
218                 pointer operator->() const { return &current->data;        }
219
220                 my_const_iterator& operator++()
```

```
221                 {
222                         current = get_next_node(current);
223                         return *this;
224                 }
225
226                 my_const_iterator operator++(int)
227                 {
228                         my_const_iterator r(*this);
229                         current = get_next_node();
230                         return r;
231                 }
232
233                 bool operator==(const my_const_iterator& i) const {        return current ==
                    ↪  i.current; }
234                 bool operator!=(const my_const_iterator& i) const {        return !(*this == i);
                    ↪  }
235
236         };
237
238         // definiamo questi member type perché sono quelli che i Container STL solitamente
            ↪  definiscono
239         using const_iterator = my_const_iterator;        // rebinding della nested class
            ↪  my_const_iterator definita sopra
240         using iterator = my_iterator;                                // rebinding della nested
            ↪  class my_iterator definita sopra
241         using value_type = T;
242
243         const_iterator begin() const
244         {
245                 return const_iterator(this);
246         }
247
248         const_iterator end() const
249         {
250                 return const_iterator(nullptr);
251         }
252
253         iterator begin()
254         {
255                 return iterator(this);
256         }
257
258         iterator end()
259         {
260                 return iterator(nullptr);
261         }
262
263 #         endif
264
265 };
266
267 // 2.c
268 // pseudo-costruttori
269 // invece di fare metodi statici facciamo funzioni templatizzate globali, così il template
    ↪  argument è inferito e diventano più comode da usare
270
271 template <class T>
```

```
tree_node<T>* lr(const T& v, tree_node<T>* l, tree_node<T>* r)
{
        return new tree_node<T>(v, l, r);
}

template <class T>
tree_node<T>* l(const T& v, tree_node<T>* n)
{
        return new tree_node<T>(v, n, nullptr);
}

template <class T>
tree_node<T>* r(const T& v, tree_node<T>* n)
{
        return new tree_node<T>(v, nullptr, n);
}

template <class T>
tree_node<T>* v(const T& v)
{
        return new tree_node<T>(v, nullptr, nullptr);
}

// 2.e

template <class T>
std::ostream& operator<<(std::ostream& os, const tree_node<T>& t)
{
        os << t.data;
        if (t.left != nullptr) os << "(" << *t.left << ")";
        if (t.right != nullptr) os << "[" << *t.right << "]";
        return os;
}

using namespace std;

// 2.d

int main()
{
        auto t1 =
                shared_ptr<tree_node<int>>(                // usiamo gli shared_ptr per non doverci ricordare di fare delete
                        // con i pseudo-costruttori globali è comodissimo costruire un albero, basta innestare le chiamate
                        lr(1,
                                lr(2,
                                        v(3),
                                        v(4)),
                                r(5,
                                        lr(6,
                                                v(7),
                                                v(8)))));

        auto t2 =
                shared_ptr<tree_node<int>>(
                        lr(1,
```

```
                                r(5,                                        // il sottoalbero destro di t2 è uguale al sinistro di t1 e viceversa
                                        lr(6,
                                                v(7),
                                                v(8))),
                                r(2,
                                        v(3),
                                        v(4))));

        // test dell'operatore di stream (<<)
        cout << "pretty printer: " << endl
                << "t1: " << *t1 << endl        // dereferenziamo per stampare perché il nostro operator<< non vuole un pointer ma un reference
                << "t2: " << *t2 << endl;

        // test dell'operatore di uguaglianza (==)
        cout << "equality: " << (*t1 == *t2) << ", " << (*t1->left == *t2->right) << endl;        // dereferenziamo gli operandi sinistro e destro del nostro operator== perchè non accetta pointer ma reference

        // test dell'iteratore non-const
        cout << "iterator: ";
        for (tree_node<int>::iterator it = t1->begin(); it != t1->end(); ++it)
        {
                int& n = *it;                // dereferenziando l'iteratore abbiamo accesso non-const al dato dentro il nodo
                n *= 2;                              // il campo data in ogni nodo può quindi essere modificato
                cout << n << " ";
        }
        cout << endl << "t1 modificato: " << *t1 << endl;        // ristampiamo t1 dopo le modifiche

        // test dell'iteratore const
        cout << "const iterator: ";
        for (tree_node<int>::const_iterator it = t1->begin(); it != t1->end(); ++it)        // t1->begin() ritorna un iterator, che viene convertito in un const_iterator dal costruttore alla linea 142
        {
                const int& n = *it;        // dereferenziando l'iteratore abbiamo accesso const al dato dentro il nodo, quindi non possiamo modificarlo ma solo leggerlo
                cout << n << " ";
        }
        cout << endl;



}
```

4.3.8 Smart Pointer (2.2.8)

```
// Questo sorgente contiene le soluzioni dell'esame scritto di PO2 del 1/7/2022 per ciò che riguarda il quesito 3, ovvero la domanda che coinvolge C++.
// I quesiti 1-2 riguardanti Java sono in un progetto IntelliJ a parte, non qui.
// Il codice qui esposto è C++14 per qualche piccolo particolare, ma in gran parte è essenzialmente vanilla.

#include <iostream>
```

```
#include <vector>

template <class T>
class smart_ptr
{
private:
        T* pt;
        ptrdiff_t offset;
        bool is_array;

        using counter_t = unsigned short;

        counter_t* cnt;

        void dec()
        {
                if (cnt != nullptr && --(*cnt) == 0)
                {
                        if (pt != nullptr)        // se non punta a nulla non c'è niente da liberare
                        {
                                if (is_array) delete pt;
                                else delete [] pt;
                        }
                        delete cnt;
                }
        }

        void inc()
        {
                if (cnt != nullptr) ++(*cnt);
        }

public:
        using value_type = T;

        smart_ptr() : pt(nullptr), offset(0), is_array(false), cnt(nullptr) {}

        explicit smart_ptr(T* pt_, bool is_array_ = false) : pt(pt_), offset(0), is_array(is_array_), cnt(new counter_t(1)) {}

        smart_ptr(const smart_ptr<T>& p) : pt(p.pt), offset(p.offset), is_array(p.is_array), cnt(p.cnt)
        {
                inc();
        }

        ~smart_ptr()
        {
                dec();
        }

        smart_ptr<T>& operator=(const smart_ptr<T>& p)
        {
                dec();
                pt = p.pt;
                cnt = p.cnt;
```

```
60                  offset = p.offset;
61                  is_array = p.is_array;
62                  inc();
63                  return *this;
64          }
65
66          // subscript
67          // l'operatore di subscript è l'unica implementazione reale; tutti gli altri operatori
            ↪  sono implementati in funzione di questo
68          // questo approccio è poco error-prone perché concentra solamente qui il calcolo esatto
            ↪  dell'indirizzo usando l'offset
69          // in altre parole, l'operatore di subscript funge da API interna a basso livello; tutto
            ↪  il resto è costruito sopra di essa
70          const T& operator[](size_t i) const
71          {
72                  return pt[offset + i];
73          }
74          T& operator[](size_t i)
75          {
76                  // capita spesso che l'implementazione const e l'implementazione non-const siano
                    ↪  identiche
77                  // in questi casi è necessario duplicare il codice, che è una pratica inelegante
                    ↪  ed error-prone
78                  // questo trucco sfrutta un giro di const cast per rimandare questa
                    ↪  implementazione a quella const appena sopra, che è l'unica che implementiamo
                    ↪  davvero
79                  // ATTENZIONE: tutto questo NON è richiesto dal tema d'esame, lo mostriamo
                    ↪  solamente a scopo didattico per insegnare una tecnica avanzata di
                    ↪  non-duplicazione del codice
80                  return const_cast<T&>(const_cast<const smart_ptr<T>&>(*this).operator[](i));
81          }
82
83          // de-reference
84          const T& operator*() const
85          {
86                  // usa l'operatore di subscript
87                  return pt[0];
88          }
89          T& operator*()
90          {
91                  // stessa tecnica per non duplicare: chiamiamo la versione const di questo
                    ↪  operatore definita qui sopra, che è l'unica che implementiamo davvero
92                  return const_cast<T&>(const_cast<const smart_ptr<T>&>(*this).operator*());
93          }
94
95          // field access
96          const T* operator->() const
97          {
98                  // anche questa implementazione sfrutta altri operatori scritti sopra: in
                    ↪  particolare usa de-reference che a sua volta usa il subscript, in questo modo
                    ↪  non richiede manutenzione
99                  return &*(*this);
100                 //      ^
101                 // si faccia attenzione a un particolare: solo l'operatore * indicato dalla
                    ↪  freccetta invoca l'overload definito da noi
102                 // il de-reference di this dentro le parentesi tonde e l'operatore & più esterno
                    ↪  invocano gli operatori nativi di C++, non i nostri overload
```

```
103         }
104         T* operator->()
105         {
106                 // altro uso della tecnica avanzata per non duplicare l'implementazione non-const
107                 // cerchiamo di capirla: lo scopo è chiamare l'implementazione const di questo
                    ↪  operatore senza duplicare il codice
108                 // 1) trasformiamo *this (che in questo scope è di tipo smart_ptr<T>&) in un
                    ↪  const smart_ptr<T>&
109                 // 2) ora che *this è castato a const, invochiamo l'operatore o il metodo che ci
                    ↪  interessa: la risoluzione dell'overload risolverà l'implementazione const,
                    ↪  non farà una ricorsione!
110                 // 3) siccome stiamo invocando la versione const, il risultato è const: ma il
                    ↪  nostro tipo di ritorno deve essere un T* non-const, pertanto bisogna
                    ↪  const-castare per TOGLIERE il const
111                 return const_cast<T*>(const_cast<const smart_ptr<T>&>(*this).operator->());
112         }
113
114         // plus
115         smart_ptr<T>& operator+=(ptrdiff_t off)
116         {
117                 offset += off;
118                 return *this;
119         }
120         smart_ptr<T> operator+(ptrdiff_t off) const
121         {
122                 // questa implementazione usa il copy-constructor e l'operator+= definito qui
                    ↪  sopra
123                 return smart_ptr<T>(*this) += off;
124         }
125
126         // minus
127         smart_ptr<T>& operator-=(ptrdiff_t off)
128         {
129                 offset -= off;        // analogo al +=
130                 return *this;
131         }
132         smart_ptr<T> operator-(ptrdiff_t off)
133         {
134                 return smart_ptr<T>(*this) -= off;
135         }
136
137         // pre
138         smart_ptr<T>& operator++()
139         {
140                 // questa implementazione usa l'operator+= e basta
141                 return *this += 1;
142         }
143         smart_ptr<T>& operator--()
144         {
145                 return *this -= 1;        // analogo al ++ ma usa il -=
146         }
147
148         // post
149         smart_ptr<T> operator++(int)
150         {
151                 smart_ptr<T> r(*this);        // copia
```

```
152                 ++(*this);                                  // pre-incrementa this: usiamo il
                     ↪  pre-incremento affinché questa implementazione dipenda totalmente
                     ↪  dall'implementazione di operator++
153                 return r;                                  // ritorna la copia
154         }
155         smart_ptr<T> operator--(int)
156         {
157                 smart_ptr<T> r(*this);
158                 --(*this);                                  // analogo a ++
159                 return r;
160         }
161
162 };
163
164 using std::string;
165
166 int main()
167 {
168         int* a = new int[5];
169         smart_ptr<int> a1(a, true), a2;
170         a2 = a1 + 10;
171         a1 -= 3;
172         a1 = ++a1 - *a2-- + a1[3];
173
174         smart_ptr<double> b(new double[10], true);
175         for (unsigned int i = 0; i < 10; ++i)
176                 b++;
177
178         string* sp = new string("ciao");
179         smart_ptr<string> s1(sp), s2;
180         s2 = s1;
181         size_t i = s2->find('a', 0);
182
183
184
185 }
```